

Variable neighborhood search for a new practical dynamic pickup and delivery problem

Junchuang Cai, Qingling Zhu, Qiuzhen Lin^{*}

College of Computer Science and Software Engineering, Shenzhen University, Shenzhen, PR China

ARTICLE INFO

Keywords:

Dynamic pickup and delivery problem
Variable neighborhood search
Heuristic search

ABSTRACT

In today's manufacturers, a large number of cargoes like materials, semi-finished products and finished products have to be delivered among factories during the manufacturing process. Thus, this paper first introduces a new Dynamic Pickup and Delivery Problem (DPDP), which has more practical constraints like dock, time windows, capacity and last-in-first-out loading. This DPDP model can fit the practical scenarios more, which cannot be directly solved by the existing optimization algorithms. To solve this problem, this paper proposes a new Variable Neighborhood Search Algorithm with Multiple local search strategies and an Efficient disturbance, called VNSME. Specifically, each new optimization period is activated when new orders are coming, in which a variable neighborhood search is used to find the best solution for this period. At each start of a period, the best solution found in the previous period is reconstructed as an initial solution for the new period by using exhaustion and cheapest insertion heuristics. Next, four different local search strategies (couple-exchange*, block-exchange*, block-relocate* and multi-relocate*) are designed to search around the initial solution, and then the currently found best solution is further perturbed by a modified 2-opt-L* method. At last, the performance of VNSME is studied on the real-world DPDP benchmarks offered by Huawei in the competition at ICAPS 2021, where the advantages of VNSME are confirmed in the competition as its final score gets the first rank among 153 teams.

1. Introduction

In today's manufacturers, a large number of cargoes like materials, semi-finished products and finished products have to be delivered among factories during the manufacturing process, which is often called a Dynamic Pickup and Delivery Problem (DPDP). Especially for the large-scale manufacturing enterprises that provide products globally, e.g., Amazon and Huawei, billions of products are delivered among hundreds of factories every year. Due to the uncertainty of requirements and production processes in the DPDP, most of the delivery requirements cannot be determined beforehand. These delivery orders are dynamically generated, which include some order information, e.g., the amount of cargoes, the time requirement, the pickup sites, and the delivery sites. These orders are completed periodically by scheduling a fleet of vehicles to do the pickup and delivery services. However, it is very difficult to find an optimal scheduling solution for the DPDP, as the delivery requests are tremendous in large-scale manufacturing enterprises.

Theoretically, the DPDP is a variant of a traditional pickup and

delivery problem (PDP) that is one of the most widely studied combinatorial optimization problems [1]. Generally, the objective of the PDP is to minimize the total travel distance or the number of used vehicles, under the requirements of customers distributed at different sites. It has been proven that the PDP is one of the NP-hard problems. In a static case, the PDP can be seen as a vehicle routing problem, in which products are picked up by vehicles from the origins and delivered to the destinations in a physical environment, without considering the new coming orders. The static PDPs arise in many applications, such as logistics, transportation systems, and robotics [2,3]. According to the surveys for PDPs [4,5], PDPs can be classified into three kinds: 1) many-to-many PDP, which has many origins and destinations for each product/customer; 2) one-to-many-to-one PDP, which delivers some products from a depot to many customers and collects other products from many customers into the depot; and 3) one-to-one PDP, which only has a given origin and a given destination for each product/customer. Most research studies [6,7] about PDPs often focus on the static case and there are a few research studies considering the dynamic counterpart of PDPs [8].

^{*} Corresponding author.

E-mail address: qiuzhlin@qq.com (Q. Lin).

<https://doi.org/10.1016/j.swevo.2022.101182>

Received 24 October 2021; Received in revised form 18 August 2022; Accepted 21 September 2022

Available online 14 October 2022

2210-6502/© 2022 Elsevier B.V. All rights reserved.

DPDPs further consider the reality of dynamically generating customer orders in PDPs. Generally, the existing approaches for solving DPDPs can be classified into two main categories. The first kind is traditional optimization methods based on operational research [14], in which the target DPDP is decomposed into a series of static problems and then each static problem can be optimized by linear programming or mixed integer programming. The other kind is the heuristic algorithms, such as cheapest insertion [9], tabu search [10], genetic algorithm [11], waiting and buffering strategy [12], and deep graph network [13], which dynamically insert new delivery orders into planned routes.

With the fast development of online shopping, a larger number of logistics transportation for products is required and DPDP variant extended from the one-to-one PDPs with various practical constraints is playing an increasingly important role in practice. One typical problem within the air cargo supply chain is a PDP variant with some constraints like time window, capacity, last-in-first-out (LIFO) loading, and dock [15], which is static and only concerns the transportation of standard unit load devices from freight forwarders' to ground handlers' warehouse. However, there are more complex and dynamic industrial scenarios in some large-scale manufacturing enterprises (e.g., Amazon and Huawei), where cargoes need to be delivered among hundreds of factories during the manufacturing process with the above constraints. In this case, orders are generated dynamically and each factory could be either pickup point or delivery point. Unfortunately, few research studies pay attention to the DPDP variant. Recently, a new benchmark [16], proposed by Huawei in the competition at ICAPS 2021, introduces a new practical DPDP variant of one-to-one PDP with the constraints of dock, time windows, capacity and LIFO loading, which makes the problem more realistic and more difficult. This benchmark set is more challenging in this field and can well reflect the practical requirements of the DPDPs as derived from a real-world application of Huawei logistics. Unfortunately, the existing approaches for the DPDPs [1,9,12,17,18] cannot solve this benchmark set well. Thus, this paper designs an effective and efficient variable neighborhood search (VNS) algorithm to solve this new DPDP variant well. As an important research area of evolutionary computation, VNS combines different local search heuristics with disturbance strategies, which has shown effectiveness on a wide variety of combinatorial optimization problems [3,19–22], such as Vehicle Routing Problems (VRPs) [23–31]. However, VNS is a generic framework, which should be appropriately instantiated for solving a specific problem. To design an effective VNS algorithm, some application domain knowledge should be employed when designing its different components, e.g., initialization, local search, and disturbance. Meanwhile, the design of VNS algorithm also has to properly balance the exploration and exploitation in the heuristic search and follows the tutorial of [32] on the design, experimentation and application of metaheuristic algorithms to real-world optimization problems.

Based on the above discussions and analyses, this paper proposes a new Variable Neighborhood Search algorithm with Multiple local search strategies and an Efficient disturbance, called VNSME, for solving the new DPDP variant. Several novel local search strategies are adopted to simultaneously enable effective exploration and efficient exploitation. Specifically, in each re-optimization step, the best solution found by our algorithm in the previous period is first restored and then a combination method of exhaustion and cheapest insertion heuristics is used to reconstruct a new solution in this period. Next, four different local search strategies, i.e., couple-exchange*, block-exchange*, block-relocate*, and multi-relocate*, are used to search around the initial solution. After that, the current best solution is disturbed by a modified 2-opt-L* method and then the found best solution is outputted finally. To study the performance of VNSME, extensive experiments have shown significant improvements of our algorithm in the practical DPDP generated from real-life scenarios of Huawei. It is noted that VNSME performs very competitively and obtains the first rank among 153 teams in the competition of ICAPS 2021 (<https://icaps21.icaps-conference.org/Competitions/>). At the same time, detailed comparison and ablation

experiments have been carried out to study the effectiveness of the proposed algorithm.

The rest of the paper is organized as follows. Section 2 provides some background information, including a brief review of the DPDPs and the motivations of this paper. Section 3 formally defines the new DPDP variant and gives its mathematical model. Section 4 first presents the framework of VNSME, followed by its detailed implementation. Section 5 presents and discusses the comparison and ablation experiments to validate the effectiveness of VNSME on the new benchmarks of Huawei. Finally, Section 6 concludes the paper.

2. Background

2.1. A brief review of the DPDPs

DPDP often needs to transport products from some origins to destinations. A categorization of DPDPs is introduced in [8] and an earlier review of PDPs can be found in [4]. DPDP widely exists in many real-life applications, such as courier operations and door-to-door transportation services. Unlike the classical VRP, there are much fewer studies on DPDP owing to its high complexity. These approaches proposed for solving DPDP in literature can be classified into the two following categories [4].

The first kind of approaches decomposes the target DPDP into a series of static problems and then optimize each static problem independently until new order requests are coming. Under such a strategy, the static optimization algorithm needs to make sure that the found solution is feasible for final decision. In [14], Savelsbergh et al. proposed a Dynamic Routing algorithm of Independent VEHICLES (called DRIVE), which decomposes the target problem into a series of static optimization problems with a subset of known delivery orders over a rolling-horizon framework. Then, a branch-and-price heuristic is presented to solve the static optimization problem, in which approximation, incomplete optimization techniques as well as a sophisticated column management scheme are used to properly balance the solution speed and solution quality in a dynamic environment. In [33], a stochastic and dynamic model for DPDP is presented by Swihart and Papastavrou, in which demands for service arrive according to a Poisson process in time. In this model, the pick-up and delivery locations of demands are independently and uniformly distributed over a service region and its optimization objective is to minimize the expected time in the system for the demands. Then, the authors gave the bounds on the performance of routing policies for the cases of using unit-capacity vehicles or multi-capacity vehicles. However, the above methods show a high computation complexity, as they need to optimize each static subproblem consecutively. Therefore, this kind of approaches cannot well fit the real-time requirement in industrial scenarios.

The second kind of approaches dynamically inserts new delivery orders into planned routes by using the heuristic algorithms with local search or prediction, which are widely used in the existing studies [4]. Regarding the hybridization of heuristic algorithms with local search, Karami et al. [34] presented a rolling-horizon based algorithm for solving the problem with same-day pickup and delivery requests for the transportation of letters and small parcels. Each of the static problems is optimized with a two-phase heuristic: the first phase uses the cheapest insertion procedure while the second one adopts a tabu search algorithm, which reduces the route costs when compared to classical (single) rolling-horizon methods. In [10], Gendreau et al. designed a tabu search heuristic to explore new solutions through a neighborhood structure based on ejection chains. Here, the heuristic aims to optimize the planned routes of vehicles in the case that new requests with the pick-up and delivery locations are generated in real-time. As validated on a set of daily instances with up to 33 requests per hour, this method can perform much better than some simple insertion methods. Moreover, Pureza and Laporte [12] presented two strategies (the waiting strategy [35] and the request buffering strategy) with a constructive-deconstructive heuristic

for solving the traditional DPDP, which can effectively reduce the lost requests and the number of vehicles. Other promising methods in this kind are also presented to deal with the application of DPDPs in a specific scenario, such as the bookings of Air taxi [36], the dial-a-flight problem about transporting passengers [37,38], and the meal delivery routing problem [39,40]. For the hybridization of heuristic algorithms with prediction, Ghiani et al. [41] proposed an anticipatory algorithm with sampling in a short-term horizon for the dynamic vehicle dispatching problem with pickups and deliveries. Solutions incorporating the incoming requests are constructed by a simple insertion method, which are further improved by a local search scheme based on simple moves. In [11], Sáez et al. proposed an adaptive predictive control algorithm with genetic algorithm and fuzzy clustering for solving a capacitated DPDP. In this method, the predictive model considers the relevant state space variables as vehicle load and departure time at stops, which helps to reduce the feasible search space, while the genetic algorithm provides near-optimal solutions for the DPDP. Moreover, Muñoz-Carpintero et al. [42] proposed a methodology based on generic evolutionary algorithm for solving the DPDP formulated under a hybrid predictive approach, in which different configurations of particle swarm optimization and genetic algorithm are considered in real time to solve the nonlinear mixed-integer optimization problem related with the hybrid predictive control approach. Here, a multi-objective optimization method is used for parameter tuning in evolutionary algorithm, which can obtain the best trade-off between optimization performance and computational effort for solving the DPDP.

2.2. Motivations

There are several important issues in the existing studies of DPDP when dealing with pickup and delivery of cargoes in real-life scenes. First, the dock constraint is not considered in the above DPDPs, which cannot be ignored during the manufacturing process in practice. In this case, vehicles in the factory need to be parked at the dock to pick up and deliver cargoes. However, the limited dock may lead to the queuing of some vehicles for uncertain times and thus some orders cannot be completed in time, which makes customers unsatisfactory with the delivery service. Second, the research of the DPDP is mainly carried out on some artificially generated data, which cannot reflect the difficulty of real-world applications. Thus, it is important to have a benchmark set containing real-world instances, which can further facilitate the research of DPDP. Third, the number of factories is usually limited in the above-mentioned scenarios. Especially, there may be many orders with the same pickup and delivery address among hundreds of thousands of orders generated at different time in a day. However, the local search process of the existing heuristic algorithms is performed based on the unit of each order, which may cause low efficiency and cannot quickly find a better solution within a limited time. If these orders on the same path can be merged in the local search process, all the pickup (delivery) nodes of these orders can be combined as a unit to perform the local search operation. Therefore, the cargoes of these orders can be loaded into the same vehicle at the pickup factory and delivered at the same destination factory.

This paper aims to address the above issues. Specifically, this paper first introduces and formulates a new model of the DPDP with the constraints of dock, time windows, capacity and LIFO loading, which is more common and realistic in the manufacturing process. Then, to better solve the DPDP with these practical constraints, this paper proposes an effective VNS algorithm with multiple local search strategies and an efficient disturbance, named VNSME. Our advantages have been validated on the real-world benchmarks of the DPDP offered by Huawei in the competition at ICAPS 2021.

3. Problem definition and mathematical model

3.1. Problem definition

The DPDP from the practical logistics scenarios is described in this section. Fig. 1 shows a simple example to illustrate the problem more intuitively. Assume that factory F_2 has only one available dock to load or unload cargoes. order o_3 is assigned to vehicle v_1 , while orders o_1 and o_2 are assigned to vehicle v_2 . vehicle v_1 first arrives at factory F_1 and then goes to the dock of factory F_1 to load cargoes. After completing the pickup of order o_3 , vehicle v_1 leaves factory F_1 and drives towards the destination factory F_2 . Before vehicle v_1 arrives at factory F_2 , vehicle v_2 has been already parked in the dock of factory F_2 to pick up cargoes, which sequentially picks up the cargoes of orders o_1 and o_2 . Since there is only one dock in factory F_2 for a vehicle to load or unload cargoes, vehicle v_1 has to wait for an available dock to unload order o_3 , until vehicle v_2 completes its orders in factory F_2 and then leaves. Subsequently, vehicle v_2 delivers order o_2 to factory F_3 and order o_1 to factory F_4 .

In summary, this DPDP variant has constraints of time windows, capacity, LIFO loading and dock. Each node (factory) has a fixed number of docks to load or unload cargoes, which means that once all the docks in a factory are occupied by vehicles, the other vehicles arriving later have to wait for the dock to be free in the queue of arrival order. Therefore, dock constraint plays a significant role in the order tardiness of the DPDP, which affects the customer's satisfaction with the service to a large extent. The order to be dispatched specifies the pickup node, the number of cargoes to be picked, loading time, unloading time and time window requirement. Within a day, there are multiple factories that can dynamically generate delivery orders and each factory could be either pickup point or delivery point. For this problem, our goal is to dispatch all orders to a fleet of vehicles, aiming to minimize the total tardiness of orders and the average traveling distance of vehicles.

3.2. Mathematical model

3.2.1. Problem parameters and decisions

The inputs of the DPDP include:

- 1 M nodes (factories) $\{F_i | i = 1, \dots, M\}$. F is a set with multiple nodes. Each node has L_d cargo docks for loading and unloading. If all docks are in use, the newly arriving vehicles must wait for their release.
- 2 A road network $G = (F, A)$, which is a complete directed graph. $A = \{(i, j) | i, j \in M\}$ is an arc set. Each arc $\langle i, j \rangle$ is associated with a non-negative transportation distance D_{ij} .
- 3 An order set $O = \{o_i | i = 1, \dots, N\}$ including each order $o_i = (F_p^i, F_d^i, Q^i, t_e^i, t_l^i)$, where F_p^i and F_d^i represent the pickup and delivery nodes, respectively, Q^i is the amount of cargoes to be delivered as represented by $Q^i = (q_{\text{standard}}^i, q_{\text{small}}^i, q_{\text{box}}^i)$ (q_{standard}^i stands for the standard pallet amount, q_{small}^i stands for the small pallet amount, and q_{box}^i stands for the box amount), t_e^i refers to the creation time and t_l^i is the committed completion time. Note that the committed completion time refers to the completion time of each order, including the vehicle traveling time, dock queuing time, dock approaching time and loading/unloading time. One standard pallet equals to two small pallets and one small pallet equals to two boxes. These orders are generated randomly.
- 4 A fleet of homogeneous vehicles $V = \{v_k | k = 1, \dots, K\}$. Each vehicle v_k has its loading capacity. The initial position for each vehicle is randomly assigned to a factory and K is the number of vehicles.
- 5 Dock approaching time T_{da} , i.e., the running time for a vehicle from the factory to the factory's dock, e.g., 30 minutes.
- 6 Loading and unloading time. For example, if there are q standard pallets in the vehicle, then the loading time $t_p = w \times q$ and unloading time $t_d = w \times q$, where $w = 180$ seconds for each standard pallet.

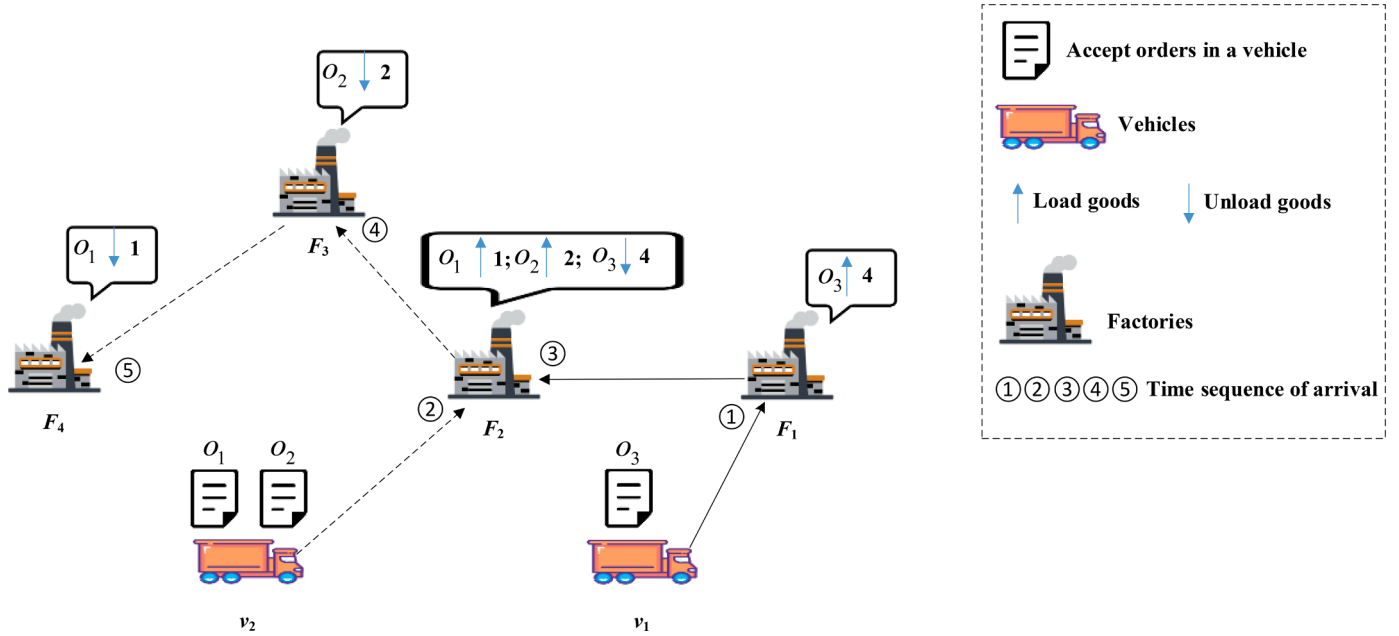


Fig. 1. A simple example of the DPDP.

The outputs of the DPDP are:

- 1 Order assignment $OA = \{(o_i, k)\}$, which consists of multiple matches between vehicles and orders. (o_i, k) represents to dispatch vehicle k to serve order o_i .
- 2 Route plan $RP = \{rp_k = \{n_1^k, n_2^k, \dots, n_{l_k}^k\} | k = 1, \dots, K\}$ for the vehicle fleet. For example, the route plan of a vehicle v_k is $rp_k = \{n_1^k, n_2^k, \dots, n_{l_k}^k\}$, where n_i^k and l_k are, respectively, the i -th factory and the number of factories that vehicle v_k should visit.

All the sets, variables and parameters of the mathematical model are collected in Table 1.

3.2.2. Objective functions

This DPDP model includes two optimization objectives. The first objective is to minimize the total tardiness of orders denoted as f_1 . If the arrival time of the order o_i to be delivered to the destination is denoted as a_d^i , then

$$f_1 = \sum_{i=1}^N \max(0, a_d^i - t_i^k) \quad (1)$$

where t_i^k is the committed completion time, and N is the total number of orders.

$$f_1 - \sum_{k=1}^K \sum_{i=1}^{l_k} \max(0, t_i^k - t_i^{y_i^k}) d_i^k = 0, \forall i \in l_k, \forall k \in V \quad (2)$$

$$t_{i+1}^k = t_i^k + w_i^k + T_{da} + t_{s,i}^k + t_{i,i+1}^k, \forall i, i+1 \in l_k, \forall k \in V \quad (3)$$

Here, (2) concretizes the first objective function in (1). f_1 is equal to the total tardiness of the orders serviced by every vehicle. The corresponding order of the i -th node visited by the vehicle v_k is denoted as y_i^k , and t_i^k is the committed completion time of the order. Specifically, the tardiness of the corresponding order is calculated when the vehicle v_k arrives at a delivery node (i.e., d_i^k is 1). Eq. (3) specifies the calculation of the arrival time for the vehicle v_k to reach the $(i+1)$ -th node, denoted as $t_{i+1}^k = t_i^k + w_i^k + T_{da} + t_{s,i}^k + t_{i,i+1}^k$. t_i^k is the arrival time for the vehicle v_k to

Table 1

All the sets, parameters, and variables of the DPDP model.

Sets	
O	Set of orders
G	Set of roads
V	Set of all homogeneous vehicles
F	Set of factories
Parameters	
D_{ij}	Travel distance between nodes i and j
L_d	The number of docks at each factory
N	The total order count
K	The number of all homogeneous vehicles
Q	Maximum loading capacity of each vehicle
T_{da}	Dock approaching time: the running time for a vehicle from the factory to the factory's dock
λ	A large positive constant
Q^i	The cargoes' amount of the order o_i : Q^i in the pickup and $-Q^i$ in the delivery of the order o_i
Variables	
x_{ij}^k	Binary variable. Its value equals to 1 if vehicle v_k moves from node i to node j
d_i^k	Binary variable. Its value equals to 1 if the i -th node visited by the vehicle v_k is a delivery node
y_i^k	The corresponding order of the i -th node visited by vehicle v_k
t_s^i	The loading or unloading time of the order o_i
q_i^k	Load of vehicle v_k when leaving node i
t_{ij}^k	Traveling time of vehicle v_k from nodes i to j
t_i^k	Time of vehicle v_k arriving at node i
w_i^k	The waiting time for vehicle v_k to wait for the dock to be allocated after reaching the node i
z_i^k	Binary variable. Its value equals to 1 if vehicle v_k is occupying a dock at node i
n_i^k	The i -th node that vehicle v_k should visit
l_k	All nodes that vehicle v_k should visit
t_e^i	The creation time of order o_i
a_d^i	The arrival time of order o_i to be delivered to the destination
t_i^k	The committed completion time of order o_i

reach the i -th node. w_i^k is the waiting time for vehicle to wait for the dock to be allocated after reaching the node i . T_{da} is the running time for a vehicle from the factory to the factory's dock. t_s^k is the time to load or unload cargoes of the order y_i^k corresponding to the i -th node visited by

vehicle v_k . $t_{i,i+1}^k$ is the time to travel from the i -th node to the $(i+1)$ -th node after the vehicle v_k finishes the service at the i -th node. Here, T_{da} for all vehicles is the same and fixed to 30 minutes.

The second objective is to minimize the average traveling distance of vehicles denoted as f_2 . If the route plan of vehicle v_k is $rp_k = \{n_1^k, n_2^k, \dots, n_{l_k}^k\}$, where n_i^k stands for the i -th factory in the route of vehicle v_k , and l_k is the total node number traveled by vehicle k . Then,

$$f_2 = \frac{1}{K} \sum_{k=1}^K \sum_{i=1}^{l_k-1} D_{n_i^k, n_{i+1}^k} \quad (4)$$

where $D_{n_i^k, n_{i+1}^k}$ is the distance from node n_i^k to node n_{i+1}^k .

The objective functions f_1 and f_2 are in conflict with each other. There is one specific scenario that many orders are generated with the same pickup and delivery nodes in different time. Then, when every order is generated and serviced at once by a vehicle, the objective function f_1 has the minimum value, while the objective function f_2 has the maximum value. On the contrary, when the earlier orders are postponed until the orders with the same pickup and delivery nodes are bundled to be served by a vehicle, the objective function f_1 has the maximum value while the objective function f_2 has the minimum value.

Then, the overall objective is:

$$\min f = \lambda \times f_1 + f_2 \quad (5)$$

where λ is a large positive constant, as business enterprises are more concerned about improving customers' satisfaction and a timely fulfillment of orders is more important in practical scenarios.

3.2.3. Constraints

This DPDP model has the following constraints:

$$q_{i+1}^k - q_i^k - Q^{k,i+1} = 0, \forall i, i+1 \in l_k, \forall k \in V \quad (6)$$

$$0 \leq q_i^k \leq Q, \forall i \in l_k, \forall k \in V \quad (7)$$

$$t_e^i \leq a_d^i \leq t_l^i, \forall i \in O \quad (8)$$

$$0 \leq t_{F_p}^k \leq t_{F_d}^k, \forall F_p^i, F_d^i \in F, \forall i \in O, \forall k \in V \quad (9)$$

$$0 \leq \sum_{k=1}^K z_i^k \leq L_d, \forall i \in F, \forall k \in V \quad (10)$$

$$x_{F_p^i, F_p^j}^k - x_{F_d^i, F_d^j}^k = 0, \forall F_p^i, F_p^j, F_d^i, F_d^j \in F, \forall i, j \in O, \forall k \in V \quad (11)$$

$$\sum_{k=1}^K \sum_{i=1}^{l_k} y_i^k d_i^k = O, \forall i \in l_k, \forall k \in V \quad (12)$$

$$y_i^k \in O, \forall i \in l_k, \forall k \in V \quad (13)$$

$$x_{ij}^k, d_i^k, z_i^k \in \{0, 1\}, \forall i, j \in F, \forall k \in V \quad (14)$$

$$n_i^k, l_k \in F, \forall k \in V \quad (15)$$

$$t_s^i, q_i^k, t_{ij}^k, t_i^k, w_i^k, t_e^i, a_d^i, t_l^i \in R+ \quad (16)$$

Constraints (6) and (7) concern vehicle capacity. Constraint (6) indicates that the cargoes' amount of the order y_{i+1}^k is the difference of the load between the vehicle v_k leaving the $(i+1)$ -th node and the i -th node. Constraint (7) guarantees that the cargoes capacity of each vehicle cannot exceed the maximum loading capacity or be less than 0. The time window for each order to be delivered to the destination is represented by constraint (8). Note that the right-hand inequality of the time window is a soft constraint, in which the penalty value (i.e., f_1) is generated when the order o_i is delivered to the destination exceeding the committed

completion time. Constraint (9) ensures that the vehicle visits the pickup node in advance of the corresponding delivery node. Constraint (10) guarantees that the number of vehicles simultaneously occupying the docks in factory F_i cannot exceed the maximum number of docks of the factory F_i . Eq. (11) models the LIFO loading strategy. Constraint (12) ensures that all orders are dispatched to the vehicles. Finally, constraints (13)–(16) define the variables.

4. The proposed VNSME algorithm

In this section, we first describe the proposed algorithm VNSME, and then introduce its three main components (i.e., initialization, local search, and disturbance) in turn. The flow chart of the proposed algorithm is illustrated in Fig. 2. Note that these components ensure that the generated solutions always satisfy the constraints of time windows, capacity, dock and LIFO loading. Moreover, if a generated solution violates the constraint of LIFO or capacity, the solution is immediately discarded. In other words, during the running of VNSME, its solution is always feasible to the DPDP instance and its fitness value is defined in (5).

4.1. General framework

Algorithm 1 clearly illustrates the overall framework of our proposed algorithm VNSME. When the new orders are dynamically generated in the current phase, VNSME first restores the solution of the previous phase from the archive and then enters the initialization process (lines 1–2) to get an initial feasible solution x_0 . Then, the local search process, including four modified local search strategies, is applied to x_0 to obtain an improved solution x_{ls} (lines 3–4). After that, VNSME enters an evolutionary process. In each iteration (lines 6–8), an improved perturbation method is used to make x_{ls} jump out of the local optimum to get a solution $x_{disturb}$, so that the local search can be used for $x_{disturb}$ to obtain a better solution x^* . Finally, save the solution x^* in the archive for the next phase (line 10).

4.2. Initialization

In the DPDP, the information is gradually revealed over time. A basic and commonly used strategy for solving a DPDP is to adapt an algorithm that can solve the static version of the problem. Two approaches can be used. The first one consists of solving a static problem each time when new information (such as a new request or a cancellation) is revealed. Under such a strategy, the static algorithm may need to be modified to ensure that its solution is feasible for decisions. One major drawback of this strategy may be its long computation time as it needs to perform a complete re-optimization every time when new information is revealed, which is inadequate for real-time setting. However, in the second approach, the static algorithm is applied only once at the beginning of the planning horizon to obtain an initial solution under the available information. When new information is coming, the current solution is updated with some heuristic methods, such as insertion heuristics, deletion heuristics, and interchange moves, sometimes coupled with a local search method. These update mechanisms are sufficiently fast to be used in a dynamic manner.

Therefore, in the initialization phase of our algorithm, we first restore the best solution of the previous period from the archive, and then use the hybrid mechanism to reconstruct a new solution for the newly coming orders. The hybrid mechanism consists of exhaustion and Cheapest Insertion (CI). A detailed description of the cheapest insertion heuristic can be found in [9,10,12,43]. Specifically, when the algorithm runs a test instance in the current period, the hybrid mechanism is first used to generate an initial solution based on all unfinished orders, and apply VNS on the initial solution to generate an improved solution x^* , which is then saved into the archive. In each subsequent period, we restore the solution of the previous period from the archive and delete

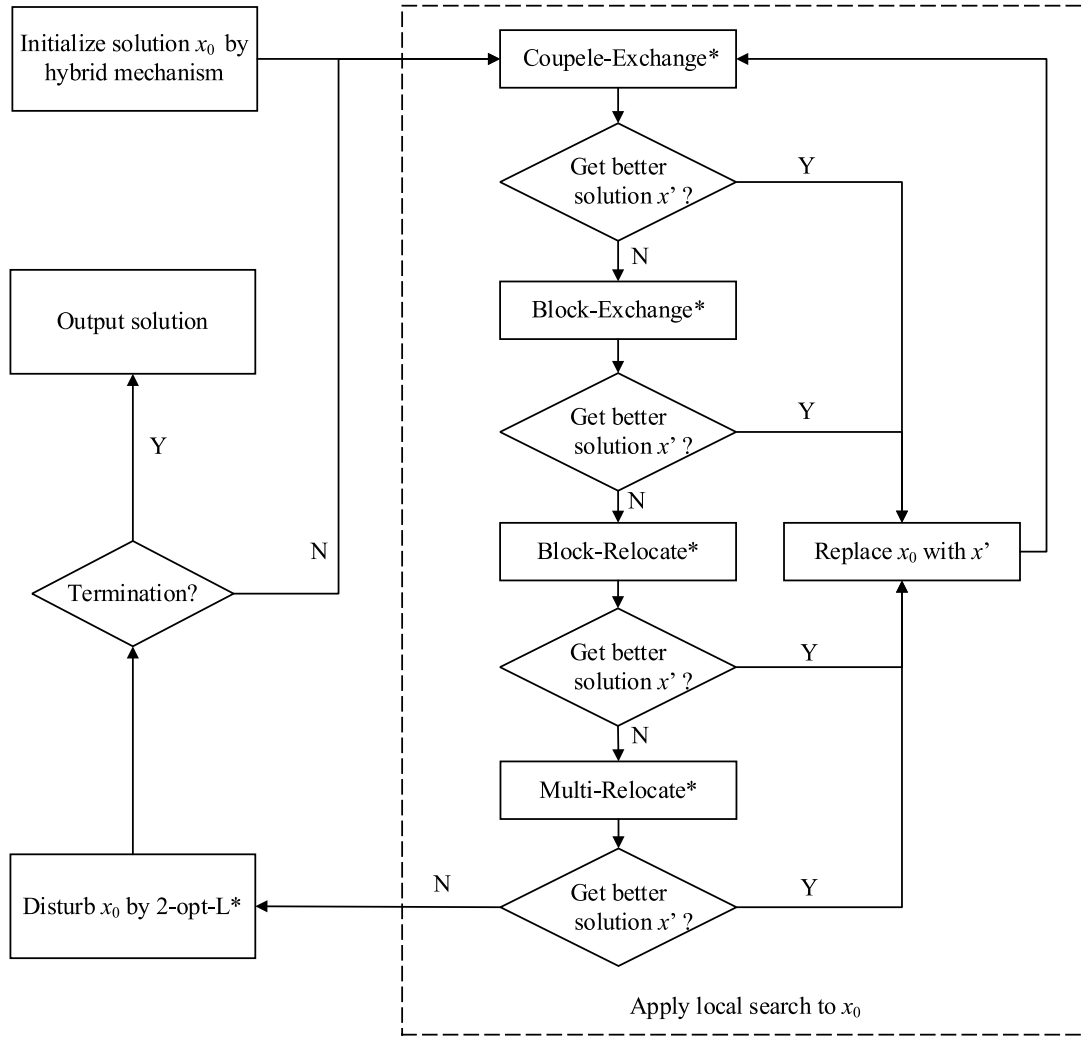


Fig. 2. The framework of the proposed algorithm.

Algorithm 1

The General Framework of VNSME.

Input: A DPDP instance
output: the final solution x^*

1. Restore the solution of the previous phase from the archive;
2. Solution $x_0 \leftarrow \text{Initialization}()$; // **Algorithm 2**
3. Solution $x_{ls} \leftarrow \text{Local_Search}(x_0)$; // **Algorithm 3**
4. $x^* \leftarrow x_{ls}$;
5. **while** $\text{used_time} < \text{MAX_TIME}$ **do**
6. Solution $x_{\text{disturb}} \leftarrow \text{Perturbation}(x_{ls})$; // **Algorithm 4**
7. Solution $x_{ls} \leftarrow \text{Local_Search}(x_{\text{disturb}})$; // **Algorithm 3**
8. $x^* \leftarrow \text{Select_Best}(x_{ls}, x^*)$; // select the best solution from x_{ls} and x^*
9. **end while**
10. Save solution x^* to the archive;
11. **Return** x^* ;

the completed nodes in the solution. Based on the restored solution in the previous period, the hybrid mechanism is only used to generate the initial solution for this period with the new orders.

As shown in **Algorithm 2**, for each new order, the hybrid mechanism traverses the planned path nodes of each vehicle after the restoring operation (lines 1–2). If the number of nodes on the planned route of vehicle v_k is less than N , exhaust all feasible routes to select the newly planned route of the vehicle v_k with the minimum f value in (5) after adding the pickup and delivery nodes of a new order (lines 3–8). Otherwise, use the CI heuristic to insert the new order into the position

Algorithm 2

Initialization().

Input: o_t : new orders in this period t ; v_k : vehicle information;
 minCost : record the minimum f value in (5) and initialized as infinite;
 N : the threshold for the number of nodes
output: x_0 : the initialized solution

1. **for** each new order o_t^i in t **do**
2. **for** each vehicle v_k , $k = 1$ to K **do**
3. **if** the number of unfinished nodes of $v_k \leq N$ **then**
4. Exhaust all feasible solutions, and keep the one solution $x_{\min}^k$ by (5);
5. **if** $x_{\min}^k < \text{minCost}$ **then**
6. $\text{minCost} \leftarrow x_{\min}^k$ and save the corresponding node sequence of v_k ;
7. **end if**
8. **else**
9. Insert the new order to v_k with the minimum value of f by (5);
10. **end if**
11. **end for**
12. Update the x_0 ;
13. **end for**

of the vehicle with the minimum f value (line 9). Among the K minimum f values generated by adding the new order to K vehicles, we select the vehicle with the minimum f value. Then, add the new order to the vehicle with the minimum f value, and update the solution (line 12).

4.3. Local search procedure

Definition 1 (Pickup and Delivery Nodes Group, PDG): If the addresses of adjacent pickup nodes and the corresponding adjacent delivery nodes are the same, respectively, these pickup nodes and delivery nodes are called Pickup and Delivery nodes Group (PDG), where the pickup nodes are called Pickup node Group, denoted by PG , and the delivery nodes are called Delivery node Group, denoted by DG , i.e., $PG \in PDG$, $DG \in PDG$. Note that if there are no two orders with the same pickup node and delivery node in a route, then the pickup node group and the delivery node group are a single pickup node and delivery node, respectively. For example, on the left part of Fig. 3, $PG_x = \{x_1^+, x_2^+\}$, $DG_x = \{x_2^-, x_1^-\}$ and $PDG_x = [PG_x, DG_x]$, while $PG_a = \{a_1^+\}$, $DG_a = \{a_1^-\}$ and $PDG_a = [PG_a, DG_a]$.

Definition 2 (Block PDG , B_{PDG}): Let $p(PG, DG)$ be the path from PG to DG in a feasible route R . We define a block $B_{PDG}(PG, DG)$ of R as the path $p(PG, DG)$. For example, on the left part of Fig. 4, $B_{PDG_x}(PG_x, DG_x) = p(PG_x, DG_x) = p(x_1^+, x_1^-)$ and $B_{PDG_a}(PG_a, DG_a) = p(PG_a, DG_a) = p(a_1^+, a_1^-)$.

As shown in lines 3–24 of Algorithm 3, the local search procedure is the core component of VNSME, which consists of four local search strategies: couple-exchange*, block-exchange*, block-relocate*, and multi-relocate*. The initial versions of these operators can be found in [44,45] and the definitions of couple and block can be found in [43]. Since the original four strategies are operated on the couple or block formed based on a single order, which may cause the algorithm to repeat a number of the same operations in a realistic scenario, where orders are delivered from an identical pickup factory to an identical delivery factory. In the case of a large number of orders, it is difficult to find a better solution in a limited time with the four original local search strategies. Therefore, we improve these four local search strategies for the new practical DPDP variant in this paper by operating in the unit of PDG or B_{PDG} each time when using one of the strategies. Take the initial path of vehicle v_1 in Fig. 3 as an example, x_1, x_2 and a_1 are the orders, and the dashed box represents the same address. The couples $[x_1^+, x_1^-]$ and $[x_2^+, x_2^-]$ are chosen to relocate (exchange with all the other couples) in the couple-relocate (couple-exchange), respectively, while only $[PG_x, DG_x]$ is selected to relocate (exchange with all the other PDG) in the multi-relocate* (couple-exchange*). Similarly, the blocks $B_{x_1}(x_1^+, x_1^-)$ and $B_{x_2}(x_2^+, x_2^-)$ are chosen to relocate (exchange with all the other blocks) in the block-relocate (block-exchange), respectively, while only a block B_{PDG_x} (i.e., B_{x_1}) defined in Definition 2 is selected to relocate (exchange with all the other B_{PDG}) in the block-relocate* (block-exchange*). These

four improved local search strategies are integrated into VNSME.

Initially, the first neighborhood couple-exchange* strategy is continuously applied to perform local search around x_0 to find an improved solution x_1 with a smaller value of f , and update it to x_0 (lines 2–6). If no improved solution is generated, the second neighborhood block-exchange* strategy is applied to perform local search around x_0 , also trying to find an improved solution of x_0 . Update x_0 if there is an improved solution, and jump back to the first neighborhood couple-exchange* strategy to continuously search for an improved solution of x_0 (lines 7–11). Otherwise, run the third neighborhood block-relocate* strategy to find an improved solution of x_0 . If an improved solution has been found, update x_0 and jump back to the first neighborhood strategy (lines 12–16). At last, run the fourth neighborhood multi-relocate* strategy to perform similar operations as before (lines 17–20). It is considered that the solution x_0 has reached the local optimum status in these four neighborhoods when an improved solution x_1 cannot be found after the fourth neighborhood strategy. Meanwhile, the time limit for the algorithm to solve a batch of orders is 10 minutes, as the orders dynamically come in each of 10 minutes. This is a hard constraint due to the practical requirement of solving the DPDP.

In a scenario where a large manufacturer needs to transport cargoes between hundreds of its factories in the production process, the pickup and delivery addresses of a large number of orders generated at different time may be the same. When using the local search methods to deal with the DPDP, if these orders are processed separately, the algorithm can be very inefficient and will occupy a lot of computation resources. Thus, no better solution can be found within an acceptable time. By considering the significant feature (PDG in Definition 1), the proposed local search strategies in our algorithm can effectively solve the new DPDP variant in this paper. The details of these local search strategies are introduced as follows.

4.3.1. The Couple-Exchange* strategy

As illustrated in Fig. 3, in the path of the vehicle v_1 , x_1^+ and x_1^- represent the pickup and delivery of an order at a certain time, respectively, while x_2^+ and x_2^- represent the pickup and delivery of another order at another time, respectively. However, the pickup address and delivery address of two orders are the same, so x_1^+ and x_2^+ with the same factory in the dashed box represent a pickup node group pg_x (i.e., $pg_x = \{x_1^+, x_2^+\}$), while x_1^- and x_2^- with the same factory in the dashed box represent a delivery node group dg_x (i.e., $dg_x = \{x_2^-, x_1^-\}$). Thus, $[pg_x, dg_x]$ is a PDG pdg_x . The meaning of the path of the vehicle v_2 in Fig. 3 is

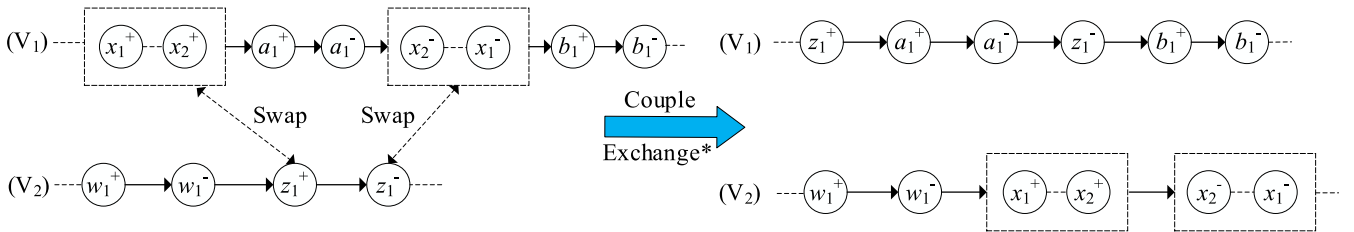


Fig. 3. An illustration of the couple-exchange* strategy (left: an initial path of vehicles v_1, v_2 ; right: new paths created by couple-exchange*).

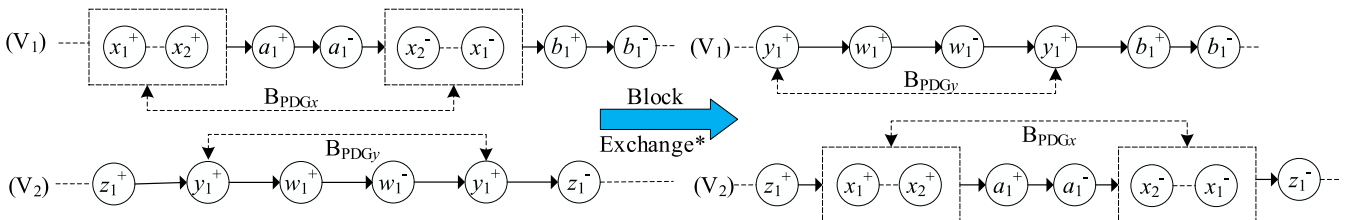


Fig. 4. An illustration of the block-exchange* strategy (left: an initial path of vehicles v_1, v_2 ; right: new paths created by block-exchange*).

Algorithm 3

The Local Search Procedure.

Input: the initialized solution x_0
output: a solution x_{ls}

```

1. Repeat
2.    $x_1 \leftarrow \text{Couple-Exchange}^*(x_0)$ 
3.   if  $f(x_1) < f(x_0)$  then
4.      $x_0 \leftarrow x_1$ ; //update the  $x_0$ 
5.   continue;
6.   end if
7.    $x_1 \leftarrow \text{Block-Exchange}^*(x_0)$ 
8.   if  $f(x_1) < f(x_0)$  then
9.      $x_0 \leftarrow x_1$ ;
10.  continue;
11.  end if
12.   $x_1 \leftarrow \text{Block-Relocate}^*(x_0)$ 
13.  if  $f(x_1) < f(x_0)$  then
14.     $x_0 \leftarrow x_1$ ;
15.  continue;
16.  end if
17.   $x_1 \leftarrow \text{Multi-Relocate}^*(x_0)$ 
18.  if  $f(x_1) < f(x_0)$  then
19.     $x_0 \leftarrow x_1$ ;
20.  continue;
21.  else
22.    break;
23.  end if
24. until  $\text{used\_time} > \text{MAX\_TIME}$ ;
25.  $x_{ls} \leftarrow x_0$ ;
26. return  $x_{ls}$ ;

```

similar to that of v_1 .

First, we obtain all available $PDGs$. Starting from the next node of the current destination node of each vehicle (because it is assumed that the destination cannot be changed after the vehicle has determined the destination), the couple-exchange* strategy traverses the path of each vehicle and stores all $PDGs$ in the archive m_1 . Next, for each PDG pdg_x (i.e., $[pg_x, dg_x] \in m_1$, traverse all other $PDGs$ pdg_z (i.e., $[pg_z, dg_z] \in m_1$, $pdg_x \neq pdg_z$ and each PDG pdg_x applies the following operation. The operator swaps the positions of pg_x with pg_z and of dg_x with dg_z , which is illustrated in Fig. 3. Note that the two $PDGs$ to be exchanged can be located on the same vehicle or on different vehicles, which ensures that the neighborhood of the solution is large enough to find a better solution. Then, compute the cost function value f in Eq. (5) of the resultant paths and implement the best swap if it has been improved upon initial paths. In the designed couple-exchange* strategy, we treat the node group as unit, which can effectively improve the efficiency of our algorithm. By using appropriate pointers, this operator runs with the worst time complexity $O(n^2)$.

4.3.2. The Block-Exchange* strategy

The block-exchange* strategy is similar to the previous one. First, we obtain all $PDGs$ in all vehicles as same as the previous one and save all corresponding blocks B_{PDGs} in the archive m_2 . The block B_{PDG} defined in Definition 2 is the sequence of nodes from PG to DG .

Next, for each block $B_{PDG_x} \in m_2$, the block-exchange* strategy traverses all other blocks $B_{PDG_y} \in m_2$ with $B_{PDG_x} \cap B_{PDG_y} = \emptyset$. Then, each

block B_{PDG_x} applies the following operation. The block-exchange* strategy swaps B_{PDG_x} and B_{PDG_y} , and computes the cost value f of the resultant paths (Fig. 4). Note that the two blocks to be swapped can be located on the same vehicle or on different vehicles, which also ensures that the neighborhood of the solution is large enough to find a better solution. Then, it implements the best swap if the resultant paths are improved, i.e., the cost function value f for all the vehicles paths is reduced. Again, this strategy requires the worst time complexity $O(n^2)$.

4.3.3. The Block-Relocate* Strategy

We first obtain all $PDGs$ in all vehicles as same as the couple-exchange* and save all corresponding blocks B_{PDGs} in the archive m_2 . For each block $B_{PDG_x} \in m_2$, the block-relocate* strategy reinserts it into all other feasible positions. The feasible position can be in the path where the block was originally located, or in other vehicle paths, which ensures that the neighborhood of the solution is large enough. This operation is illustrated in Fig. 5. After applying the block-relocate* strategy, the cost function value f of the resultant paths is computed. Then, the best relocation is implemented if the resultant paths are improved, i.e., the cost function value f is reduced. This operator requires the worst time complexity $O(n^2)$.

4.3.4. The Multi-Relocate* strategy

The multi-relocate strategy [45] is a local search operator, which is derived from the relocate-couple strategy [44]. For each couple $[x^+, x^-]$ in all routes, the relocate-couple strategy computes the best positions to relocate x^+ and x^- . However, this information is saved only if $[x^+, x^-]$ is the best couple to relocate, and the operator relocates only the best couple identified. This implies that all the information computed by relocate-couple about the other couples is lost after relocating the best one. However, it may save this information for further use. The idea behind the multi-relocate strategy is to save every couple $[x^+, x^-]$ in a queue whose relocation produces a new better solution, to relocate the best couple identified, and then to attempt to relocate as many couples as possible from the queue in the new solution. Note that when the multi-relocate strategy relocates only the best couple, this solution is exactly the same as the solution produced by the relocate-couple strategy.

The multi-relocate* strategy used in our algorithm is improved from the multi-relocate strategy. The search process for the former is similar to the latter except that each time the latter relocates a couple and the former relocates pdg . As shown in Fig. 6, the two dashed boxes of v_1 , composed of two orders with the same pickup and delivery address, are $pdg[pg_x, dg_x]$, which are relocated in the multi-relocate* strategy.

We first obtain all $PDGs$ in all vehicles as same as the couple-exchange* strategy. The idea behind the multi-relocate* strategy is also to save every pdg_i $[pg_i, dg_i]$ in a queue whose relocation produces a new better solution, to relocate the best PDG identified, and then to relocate as many $PDGs$ as possible from the queue in the new solution. The detailed operation can refer to the multi-relocate strategy [45]. This operator requires the worst time complexity $O(n^3)$.

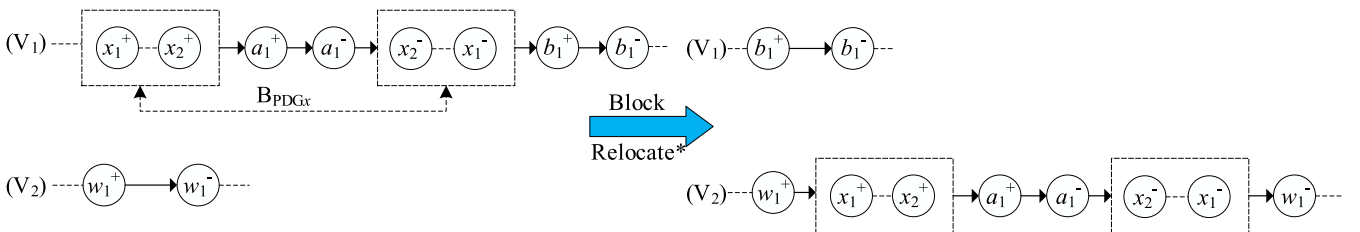


Fig. 5. An illustration of the block-relocate* strategy (left: an initial path of vehicles v_1 , v_2 ; right: new paths created by block-relocate*).

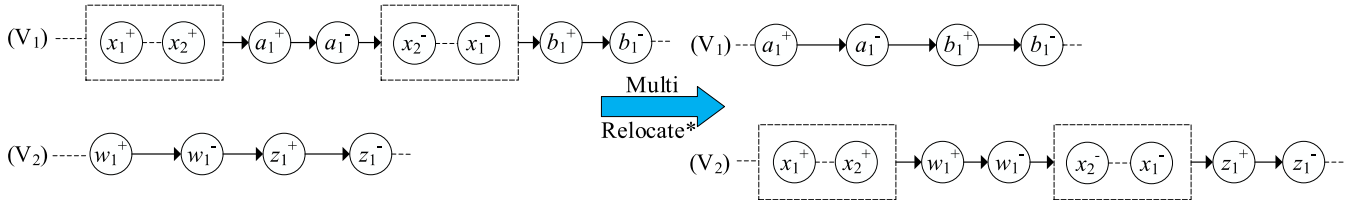


Fig. 6. An illustration of the multi-relocate* strategy (left: an initial path of vehicles v_1, v_2 ; right: new paths created by multi-relocate*).

4.4. Perturbation operator

Although the above-described local search strategies can achieve transformation and improvement within and between routes, the solution may still fall into the local optimum. The perturbation operator is needed to make solution x_{ls} jump out of the local optimum, so that the local search strategies can be used again to find a better solution. Once a local optimum has been found in the region, an operator with a bigger step size is needed for jumping out of it [33]. We propose a 2-opt-L* operator to achieve this goal. This operator is an improvement of 2-opt-L [32]. As shown in lines 1–8 of Algorithm 4, the solution obtained by the 2-opt-L* disturbance is accepted when its cost function value f is less than $f(x_{ls}) \times \delta$, which ensures that this new solution can jump out of the local optimal region. Note that $f(x_{ls})$ is the cost function value f of the best solution x_{ls} found by the local search so far, and δ is the disturbance parameter to ensure that the new solution can jump out of the local optimum.

5. Experimental study

In this section, we first introduce a new DPDP benchmark with specific constraints based on Huawei's real-world scenarios in detail. We name the new DPDP benchmark set as HW benchmarks. Next, comprehensive comparison experiments are conducted to evaluate the effectiveness of VNSME. After that, we perform a comprehensive ablation study to assess the contributions of the novel components integrated into VNSME. Finally, the effect of different values of the parameter δ (the disturbing degree) is investigated. The source code of VNSME, as well as all the benchmark sets used in the experiments, are available at <https://competition.huaweicloud.com/information/1000041411/Winning>.

5.1. Benchmark set

5.1.1. Description of the DPDP benchmarks

To the best of our knowledge, the research of the DPDP is mainly carried out on the artificial dataset and no test instances are available for the new DPDP variant studied in this article. However, in the real-world scenario, the dock constraint of each node (factory) is inevitable in the logistics field, as the dock constraint may cause the queue waiting for the dock to be free for loading and unloading cargoes, which has a significant impact on whether the order can be completed within the committed completed time. Besides, artificially generated data cannot

well reflect the difficulty of real-world applications. Therefore, this new DPDP variant is a more realistic problem in the logistics domain. Then, a new instance set (HW benchmarks) proposed at ICAPS 2021 is introduced to promote the research for the real-life application, which can serve as a new and more challenging benchmark set in this field.

The HW benchmarks are generated from the real-life scenarios of Huawei Technologies Ltd. A large amount of cargoes need to be delivered among factories during the manufacturing process and many features are shown in the scenarios. First, most customers' delivery requirements become uncertain and cannot be fully decided beforehand, which reflects the dynamic characteristics. Besides, a number of homogeneous vehicles are periodically scheduled to deliver cargoes picked up from the source factories to the target factories to serve the orders. Many constraints are considered from the real-life scenarios, i.e., both pickup and delivery operations must be executed in the predefined time windows to provide satisfactory service, the subsequent vehicles need to queue up and wait for the dock to be free when all the docks of a factory are occupied, and vehicles loaded with products meet FILO and capacity constraints.

The datasets from Huawei's real system contain 30 days of historical data, with 2000~5000 orders and 100~200 vehicles per day. As illustrated in Table 2, the HW benchmarks contain four homogeneous vehicle quantity scales (5, 20, 50 and 100) and eight order quantity scales (50, 100, 300, 500, 1000, 2000, 3000, and 4000). Specifically, the HW benchmarks contain 8 problem scales, which are classified according to the number of vehicles and orders. For each problem scale, it contains eight instances, which finally has a benchmark set of 64 instances. The detailed generations of each problem scale are as follows. Take the largest problem scale (HW57 - HW64) as an example. For each instance, a day is first randomly selected from the sampled 30 days. Then, 4000 orders, randomly selected from the historical data of the selected day, are served by 100 vehicles. The generations of the other problem scales are similar to the generation of the largest problem scale. Therefore, the HW benchmarks have a marked distinct distribution. In addition, there are 154 factories where hundreds of delivery orders are generated every day in this real-life scene. The distance between any pair of factories in the environment is identical to that in the real world. Traveling time condition is simplified by considering a constant vehicle average speed when moving from one factory to another. A number of homogeneous vehicles are initialized at various factories following the actual geographical distribution of vehicles. Further, in order to facilitate the description of the comparison experiments, we divide these 64 instances into three more general problem scales (small, medium, and

Algorithm 4

Perturbation Procedure.

Input: solution x_{ls}
output: the perturbation solution $x_{disturb}$

1. $x_0 \leftarrow x_{ls}$
2. $x_{disturb} \leftarrow x_{ls}$
3. **for** each route $r_i \in x_{ls}$ **do**
4. $x_{ls} \leftarrow \min\{2\text{-opt-L}^*(r_i)\}$;
5. **if** $f(x_{ls}) < f(x_0) \times \delta$ **then** // $\delta > 1$
6. $x_{disturb} \leftarrow x_{ls}$
7. **end if**
8. **end for**
9. **return** $x_{disturb}$;

Table 2

The number of vehicles and orders of different scale instances in the benchmark set.

Instance	Vehicle number	Order Number	Scale
HW1 - HW8	5	50	small
HW9 - HW16	5	100	small
HW17 - HW24	20	300	small
HW25 - HW32	20	500	small
HW33 - HW40	50	1000	medium
HW41 - HW48	50	2000	medium
HW49 - HW56	100	3000	large
HW57 - HW64	100	4000	large

large) according to the number of orders. In order to provide customers with satisfactory service, the HW benchmark aims to minimize the function value f_1 in (1), i.e., the total tardiness of orders. So λ takes a sufficiently large positive number to ensure that f_1 is prior to f_2 in (4).

5.1.2. Distribution of the New DPDP benchmarks

Since the actual traveling time of vehicles under the dynamic

environment cannot be obtained offline, a simulation environment provided by Huawei is applied to show the dispatching process within Huawei's real-world scenarios. The simulator with the result-checking method is provided to evaluate the performance of our algorithm over the planning horizon (e.g., one day). In the simulator, a day is split into time intervals of the same length (each time interval has 10 minutes).

The HW benchmark set is divided into eight problem scales. Besides,

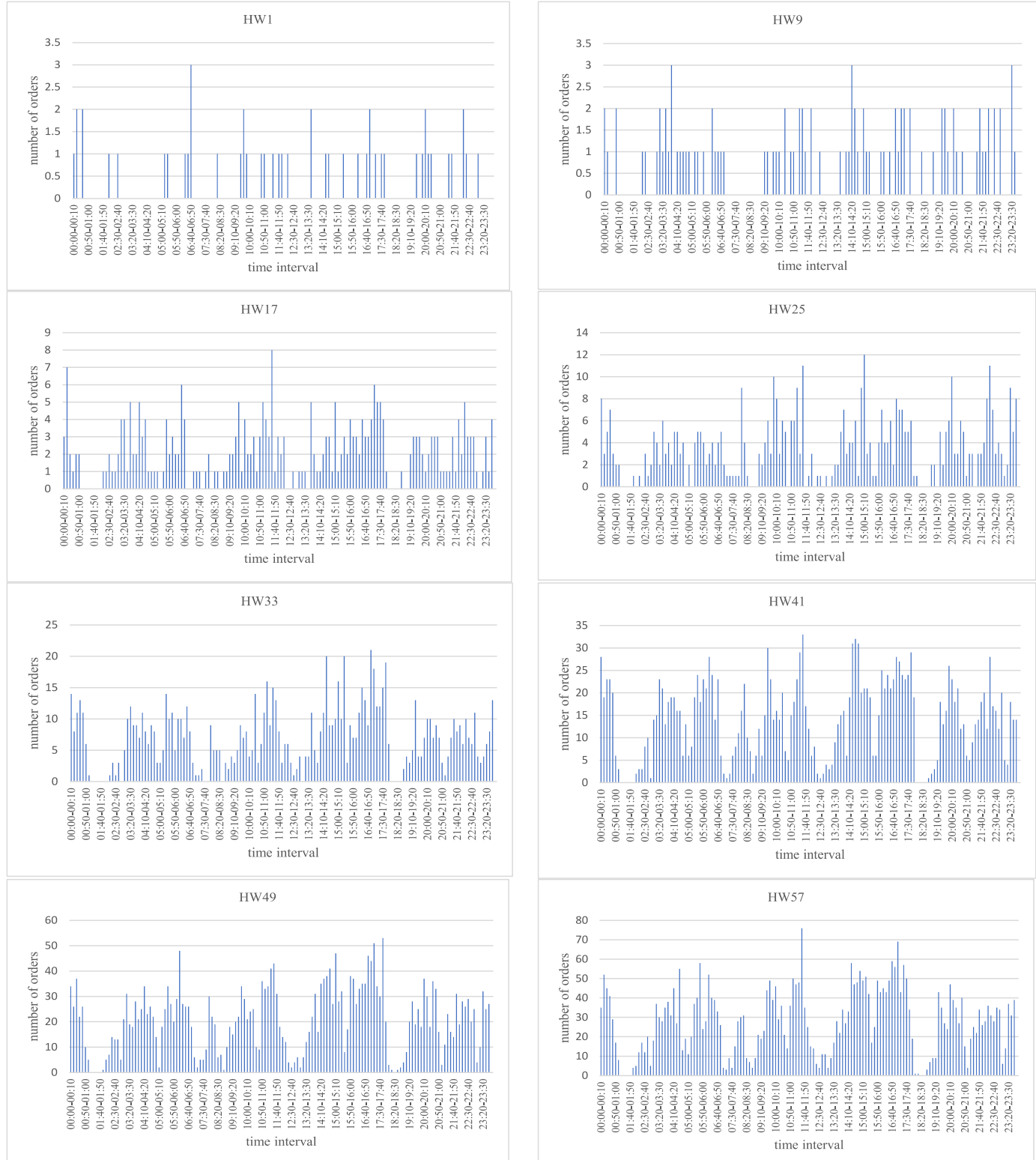


Fig. 7. The distributions of new orders generated by the first instance of each problem scale in the HW benchmark set.

the eight instances of each scale problem have the same number of vehicles and orders, in which the distribution of new orders for each instance in the same problem scale is similar. Therefore, Fig. 7 clearly shows the distributions of new orders generated dynamically in one day by the first instance of each problem scale in the HW benchmark set. As can be seen from Fig. 7, the four smallest problem scales (HW 1-8, HW 9-16, HW 17-24 and HW 33-40) have sparse distributions of new orders in 144 intervals in a day and the number of new orders that appear in each interval is small, in which the exhaustive method of the initialization strategy introduced in Section 4.2 can handle these small-scale problems well. The four larger problem scales, i.e., HW 33-40, HW 41-48, HW 49-56 and HW 57-64, distributed very differently from the previous four ones. Their new orders are densely distributed at 144 intervals of a day, and the number of new orders is large at most intervals. The order numbers for these four larger scale problems are 1,000, 2,000, 3,000, and 4,000, respectively. These numerous orders for each instance are generated in 154 factories at 144 intervals of a day, so there are many orders with the same pickup and delivery address. However, the latest local search strategies [42] are usually run on the couple or block formed based on a single order. Thus, they consume redundant computation resources, which are inefficient and also difficult to find promising solutions. The proposed four local search strategies are based on the PDG or B_{PDG} , which can improve the search efficiency of our algorithm and are easier to find more promising solutions.

5.2. The compared algorithms and their parameter settings

When solving the new DPDP variant using the dataset from real-world scenes, VNSME has shown much excellent performance. In the DPDP competition held by Huawei Technologies Ltd at ICAPS 2021 [43], the performance of VNSME is significantly better than the compared algorithms developed by other 153 teams on the above benchmark set generated from real-world scenarios of Huawei. In the competition, the submitted algorithms are evaluated on the hidden benchmark dataset, which contains a few days of historical data, with 2000~5000 orders and 100~200 vehicles per day. The hidden benchmark dataset and the HW benchmarks mentioned in Section 5.1 are all generated from real-world scenes but on different dates.

To comprehensively study the performance of VNSME, VNSME is compared with two state-of-the-art heuristic algorithms (i.e., the silver-winning and bronze-winning algorithms in ICAPS 2021 [46]) and a traditional baseline on the HW benchmarks and the hidden benchmark on the Huawei Cloud platform. These three compared algorithms are briefly introduced as follows.

- The silver-winning algorithm: This algorithm adopts a strategy to transform the DPDP problem into a knapsack problem and packs nodes for each route taking all practical constraints into consideration at the same time. Specifically, it is developed from the perspective of threshold check, i.e., allocating the orders by checking whether they reach the threshold of delivery time and vehicle capacity (if so, those orders would be allocated). Besides, some trick strategies like assigning the orders to the incoming vehicles are implemented. It also dispatches the orders based on some specific rules like urgency rank of unallocated orders.
- The bronze-winning algorithm: This algorithm applies the CI method [9] to construct an initial solution for the DPDP, in which the pickup node and delivery node are consecutive in the route for every insertion operation. Next, the solution is improved by local search through a ruin-reconstruct strategy.
- The baseline algorithm: This algorithm solves the DPDP by the traditional method dealing with static PDP [14]. In every optimization period, it inserts the pickup and delivery nodes of all uncompleted orders into the available vehicles adjacently to generate a feasible solution.

Table 3

The settings of the comparison experiments.

Parameter	Description	Value
λ	Penalty value for one hour tardiness to ensure user satisfaction	10000/ 3600
Dock number	The maximum number of docks in each factory	6
Dock approaching time	The time for a vehicle between arrival at the factory and the allocation of the dock without considering queuing	1800 seconds
MAX_TIME	The maximum time limit for the algorithm to get a solution at each optimization period of the DPDP	600 seconds

The running time of all the compared algorithms and VNSME to process the orders in each time interval is limited to 10 minutes. Each instance of the HW benchmark is run ten times independently by each algorithm, taking the best result as its best approximate optimum of the instance. The source codes of VNSME, the silver-winning algorithm, and the bronze-winning algorithm can be found from the official competition website [47], and the baseline algorithm is available in [48]. These algorithms have been tuned to the best by their authors for solving the same set of problems adopted in this paper, which can ensure a fair comparison. The detailed settings of our experiments are summarized in Table 3, which are consistent with the settings of the DPDP competition at ICAPS 2021.

The comparison experiments on the HW benchmarks are run in Ubuntu 18.04.4 LTS operation system, Intel(R) Core (TM) i5-9500 CPU @ 3.00GHz $\times$ 4, and 21G memory. The comparison experiments on the hidden benchmarks are run in the Huawei Cloud platform docker, where the configuration is Ubuntu 18.04.4 LTS operation system, Intel(R) Xeon (R) Gold 6151C CPU @ 3.00GHz CPU, and 32G memory.

5.3. The comparison experiments on the HW benchmarks

In this section, the comparison experiments between VNSME and the silver-winning, bronze-winning and baseline algorithms on small-scale, medium-scale and large-scale problems of HW benchmarks are introduced. The criterion for judging the effectiveness of these algorithms is the f value in Eq. (5) (i.e., $\lambda \times f_1 + f_2$), where f_1 in Eq. (1) represents the total tardiness of orders, f_2 in Eq. (4) represents the average traveling distance of vehicles, and λ is a penalty value for one hour tardiness of orders to ensure the priority of providing customer-satisfied services.

To investigate VNSME's effect when λ in Eq. (5) is unnecessarily large, we also conduct the experiments on another overall objective ($\min f = f_1 + \lambda \times f_2$) in small-scale, medium-scale and large-scale instances of the HW benchmarks. All the other parameter values are set the same as the previous experiments. It means that the traveling distance of vehicles has a higher priority than minimizing the tardiness of orders.

5.3.1. Results on small-scale HW benchmarks

For the original overall objective in Eq. (5), Table 4 presents the best and average performance of VNSME and other three compared algorithms on the small-scale HW benchmarks. The column headed "PDB" in Table 4 presents the percentage deviation from the reference value of the best solution, which is computed as $(f_{best} - f_{ref})/f_{ref}$ (f_{best} is the best solution obtained by an algorithm on an instance across 10 runs while f_{ref} is the best solution of the instance found by all the compared algorithms across 10 runs). Since the DPDP in our paper is a problem in the real industrial scenario, we take the f_{ref} obtained by all the compared algorithms as the approximate optimal value for each instance. It is worth mentioning that, on 26 out of 32 instances, VNSME found the best solution in the 10 independent runs and performed significantly better than the other three algorithms from the perspective of the PDB. Besides, the average performance $Avg(f)$ on the small-scale instances of VNSME still outperforms the others.

Specifically, as illustrated in Supplementary Table 1 of the

Table 4

Comparison results of VNSME, silver-winning, bronze-winning and baseline algorithms on the small-scale instances of the HW benchmarks in ten runs (f_1 represents the total tardiness of orders, f_2 represents the average traveling distance of vehicles, $\lambda=10000/3600$, and PDB represents the percentage deviation from the reference value of the best solution).

instance	VNSME		Silver		Bronze		baseline	
	$\lambda \times f_1 + f_2$	PDB	$\lambda \times f_1 + f_2$	PDB	$\lambda \times f_1 + f_2$	PDB	$\lambda \times f_1 + f_2$	PDB
HW1	1.34E+02	0.033	2.30E+03	16.66	1.30E+02	0.000	1.58E+05	1212
HW2	9.56E+01	0.046	3.05E+04	332.6	9.14E+01	0.000	8.98E+04	981.4
HW3	9.68E+01	0.003	3.58E+04	369.9	9.65E+01	0.000	3.38E+04	349.2
HW4	9.46E+01	0.000	5.49E+03	57.04	1.04E+02	0.100	4.18E+04	440.9
HW5	3.31E+03	0.000	1.69E+04	4.112	5.45E+03	0.649	1.47E+05	43.46
HW6	1.05E+02	0.000	4.76E+03	44.31	1.18E+02	0.123	5.24E+04	497.8
HW7	4.39E+03	0.000	1.28E+04	1.916	7.36E+03	0.677	9.57E+04	20.80
HW8	6.88E+01	0.000	7.97E+02	10.57	7.69E+02	10.17	3.88E+04	562.6
HW9	1.52E+02	0.000	1.81E+05	1188	8.48E+03	54.70	2.12E+06	13926
HW10	1.94E+05	0.039	1.77E+06	8.465	1.87E+05	0.000	5.42E+06	27.98
HW11	1.98E+02	0.000	1.81E+05	914.2	3.04E+03	14.37	1.92E+06	9707
HW12	5.29E+04	0.000	5.67E+05	9.709	8.42E+04	0.590	3.25E+06	60.38
HW13	7.18E+03	24.91	2.20E+05	793.2	2.77E+02	0.000	2.50E+06	9024
HW14	9.39E+03	0.201	2.37E+05	29.30	7.82E+03	0.000	2.61E+06	332.7
HW15	1.35E+04	0.000	7.93E+05	57.54	1.49E+05	9.999	3.36E+06	247.0
HW16	1.68E+04	0.000	8.54E+05	49.95	5.78E+04	2.449	3.24E+06	192.3
HW17	8.17E+01	0.000	9.65E+01	0.181	4.11E+02	4.029	1.31E+06	16028
HW18	8.22E+01	0.000	1.12E+02	0.363	8.56E+03	103.1	9.72E+05	11825
HW19	1.09E+02	0.000	4.78E+02	3.403	4.15E+03	37.22	1.93E+06	17778
HW20	3.30E+03	0.000	3.98E+03	0.207	1.62E+04	3.915	2.10E+06	636.0
HW21	1.13E+02	0.000	2.34E+03	19.64	1.85E+04	162.2	2.60E+06	22941
HW22	1.65E+03	0.000	3.32E+03	1.018	2.11E+04	11.82	2.29E+06	1390
HW23	1.05E+02	0.000	6.87E+03	64.19	8.09E+02	6.678	2.32E+06	22016
HW24	9.44E+01	0.000	1.29E+03	12.66	2.00E+03	20.18	1.44E+06	15250
HW25	1.03E+04	0.000	9.24E+04	8.009	2.15E+04	1.096	3.69E+07	3596
HW26	8.95E+03	0.000	2.75E+05	29.73	5.40E+04	5.035	3.54E+07	3955
HW27	1.34E+02	0.000	1.84E+04	136.2	1.88E+04	139.2	3.13E+07	233459
HW28	7.10E+03	0.000	9.44E+03	0.329	1.49E+04	1.098	2.86E+07	4026
HW29	6.40E+03	0.000	1.63E+05	24.47	4.24E+04	5.627	3.64E+07	5687
HW30	1.19E+02	0.000	7.40E+04	621.1	2.11E+04	176.4	3.19E+07	268212
HW31	2.26E+04	0.000	1.47E+05	5.505	2.33E+04	0.031	3.80E+07	1680
HW32	7.67E+03	0.000	6.04E+04	6.874	1.96E+04	1.555	3.63E+07	4731
	26/32		0/32		6/32		0/32	
Avg(f)	1.16E+04		1.80E+05		2.50E+04		9.84E+06	
PDB(Avg)	0.000		14.53		1.151		846.6	

supplementary material, when comparing our VNSME with the silver-winning, bronze-winning and baseline algorithms in small-scale problems of HW benchmarks (i.e., HW1-HW32), the experimental results show that our algorithm has significant advantages in minimizing the total tardiness of orders or the average traveling distance of every vehicle. Moreover, the solutions obtained by VNSME are better than the other three compared algorithms on f_1 among the 32 small-scale instances of HW benchmarks. Especially, VNSME has several obvious advantages over the other three algorithms in most small-scale instances, such as HW8, HW9 and HW11 to HW19. Although VNSME is more concerned about minimizing the total tardiness of orders f_1 to provide better transportation services, it still shows the better performance in minimizing the distance of vehicles in small-scale HW benchmarks, which is confirmed by the fact that VNSME outperforms the other compared algorithms at least in 19 out of 32 small-scale HW benchmarks. Meanwhile, it is observed that the performance of the baseline algorithm on the small-scale HW benchmarks is significantly worse than the other algorithms, which proves once again that the heuristic algorithm is superior to the traditional method that solves the DPDP problem by reprocessing the PDP in each separate period. In summary, our algorithm is of high efficiency when dealing with small-scale HW benchmarks.

As for the experiments on another overall objective ($\min f = f_1 + \lambda \times f_2$), the compared results on the small-scale instances of the algorithms are illustrated in Table 5, which shows that VNSME outperforms the other compared algorithms on 30 out of 32 instances in terms of the best solution in 10 independent runs. Comparing to the other three compared algorithms, the average performance of VNSME is still significantly

better. Although VNSME has not gained the best solution in the second and thirteenth instances, their PDB values to the best result are still small, which indicates that VNSME performs both strongly and robustly even if reversing the priority of minimizing f_1 and f_2 .

5.3.2. Results on medium-scale HW benchmarks

As for the original overall objective in Eq. (5), the best and average performance of VNSME and the three compared algorithms on medium-scale HW benchmarks are presented in Table 6. The efficacy of VNSME can be evaluated from two perspectives, i.e., the best and the average performance obtained by VNSME in 10 independent runs. VNSME found the best solution on 15 out of 16 instances while only the silver-winning algorithm can find one best solution, which outperforms the other three algorithms significantly from the perspective of the PDB. What is more, the average f values of VNSME among HW33-HW48 are evidently better than the corresponding values obtained by the other three algorithms. More specifically, as illustrated in Table 3 of the supplementary material, it can be obviously concluded that VNSME significantly outperforms the other three algorithms in minimizing f_1 , f_2 and f on the medium-scale HW benchmarks. VNSME found the solutions with the smallest values of f_1 on 14 out of 16 medium-scale HW instances, while only the silver-winning algorithm can find two solutions with the smallest values of f_1 . Meanwhile, although VNSME focuses on reducing f_1 to minimize the objective value f of the practical DPDP, it still outperforms the other three compared algorithms in searching for the better solution with a smaller value of f_2 on 10 out of 16 medium-scale HW instances. These results show that, on medium-scale HW instances, our algorithm can prioritize improving user's satisfaction while also

Table 5

Comparison results of VNSME, silver-winning, bronze-winning and baseline algorithms on the small-scale instances of the HW benchmarks in ten runs (f_1 represents the total tardiness of orders, f_2 represents the average traveling distance of vehicles, $\lambda=10000/3600$, and PDB represents the percentage deviation from the reference value of the best solution).

instance	VNSME		Silver		Bronze		baseline	
	$f_1 + \lambda \times f_2$	PDB	$f_1 + \lambda \times f_2$	PDB	$f_1 + \lambda \times f_2$	PDB	$f_1 + \lambda \times f_2$	PDB
HW1	3.60E+02	0.000	1.24E+03	2.448	3.62E+02	0.005	5.74E+04	158.3
HW2	2.66E+02	0.046	1.12E+04	43.25	2.54E+02	0.000	3.27E+04	127.9
HW3	2.62E+02	0.000	1.33E+04	49.64	2.68E+02	0.024	1.26E+04	47.09
HW4	2.62E+02	0.000	2.31E+03	7.830	2.89E+02	0.105	1.54E+04	57.87
HW5	1.41E+03	0.000	6.43E+03	3.573	2.19E+03	0.561	5.35E+04	37.03
HW6	2.92E+02	0.000	2.08E+03	6.125	3.28E+02	0.124	1.93E+04	65.12
HW7	1.92E+03	0.000	5.02E+03	1.619	3.00E+03	0.563	3.50E+04	17.24
HW8	1.91E+02	0.000	5.02E+02	1.631	4.36E+02	1.285	1.43E+04	73.87
HW9	4.45E+02	0.000	6.58E+04	146.6	3.50E+03	6.855	7.65E+05	1715
HW10	4.82E+04	0.000	6.37E+05	12.20	6.79E+04	0.408	1.95E+06	39.44
HW11	5.00E+02	0.000	6.57E+04	130.3	1.59E+03	2.169	6.92E+05	1382
HW12	1.95E+04	0.000	2.05E+05	9.474	3.07E+04	0.573	1.17E+06	58.90
HW13	9.92E+02	0.786	7.99E+04	142.8	5.55E+02	0.000	8.99E+05	1618
HW14	2.63E+03	0.000	8.58E+04	31.64	3.20E+03	0.219	9.42E+05	357.3
HW15	5.39E+03	0.000	2.86E+05	52.07	5.41E+04	9.033	1.21E+06	223.6
HW16	6.47E+03	0.000	3.08E+05	46.61	2.13E+04	2.289	1.17E+06	179.3
HW17	2.33E+02	0.000	2.68E+02	0.150	3.56E+02	0.529	4.72E+05	2026
HW18	2.33E+02	0.000	3.10E+02	0.332	3.30E+03	13.190	3.51E+05	1505
HW19	3.15E+02	0.000	4.87E+02	0.548	1.76E+03	4.579	6.96E+05	2208
HW20	1.49E+03	0.000	1.77E+03	0.187	6.08E+03	3.075	7.57E+05	506.3
HW21	2.94E+02	0.000	1.17E+03	2.981	6.93E+03	22.539	9.38E+05	3185
HW22	8.23E+02	0.000	1.45E+03	0.767	7.82E+03	8.498	8.26E+05	1002
HW23	3.08E+02	0.000	2.82E+03	8.145	5.61E+02	0.820	8.35E+05	2708
HW24	2.53E+02	0.000	7.43E+02	1.940	9.58E+02	2.792	5.18E+05	2051
HW25	2.48E+03	0.000	3.37E+04	12.62	8.07E+03	2.259	1.33E+07	5369
HW26	4.49E+03	0.000	9.97E+04	21.17	1.98E+04	3.409	1.27E+07	2834
HW27	3.51E+02	0.000	7.06E+03	19.13	7.09E+03	19.213	1.13E+07	32124
HW28	2.83E+03	0.000	3.75E+03	0.327	5.64E+03	0.997	1.03E+07	3641
HW29	2.88E+03	0.000	5.91E+04	19.54	1.56E+04	4.418	1.31E+07	4549
HW30	3.36E+02	0.000	2.70E+04	79.47	7.91E+03	22.532	1.15E+07	34123
HW31	4.53E+03	0.000	5.35E+04	10.80	8.76E+03	0.935	1.37E+07	3017
HW32	3.08E+03	0.000	2.22E+04	6.187	7.39E+03	1.396	1.31E+07	4241
30/32			0/32		2/32		0/32	
Avg(f)			6.53E+04		9.31E+03		3.54E+06	
PDB(Avg)			17.31		1.612		992.7	

Table 6

Comparison results of VNSME, silver-winning, bronze-winning and baseline algorithms on the medium-scale instances of the HW benchmarks in ten runs (f_1 represents the total tardiness of orders, f_2 represents the average traveling distance of vehicles, $\lambda=10000/3600$, and PDB represents the percentage deviation from the reference value of the best solution).

instance	VNSME		Silver		Bronze		baseline	
	$\lambda \times f_1 + f_2$	PDB	$\lambda \times f_1 + f_2$	PDB	$\lambda \times f_1 + f_2$	PDB	$\lambda \times f_1 + f_2$	PDB
HW33	1.59E+03	0.0000	7.03E+03	3.4263	9.50E+04	58.815	3.86E+07	24302
HW34	1.05E+04	0.0000	1.05E+04	0.0005	5.44E+04	4.1834	3.98E+07	3791.2
HW35	3.44E+03	0.0000	1.58E+04	3.5909	1.04E+05	29.218	5.11E+07	14846
HW36	1.75E+04	0.0000	2.32E+04	0.3242	1.13E+05	5.4496	4.67E+07	2664.4
HW37	1.12E+04	1.1845	5.14E+03	0.0000	9.57E+04	17.618	4.67E+07	9084.6
HW38	1.50E+04	0.0000	3.01E+04	1.0008	9.42E+04	5.2615	4.70E+07	3123.1
HW39	1.49E+04	0.0000	2.23E+04	0.4947	8.52E+04	4.7107	4.43E+07	2968.2
HW40	1.02E+04	0.0000	2.43E+04	1.3819	1.17E+05	10.468	3.73E+07	3655.1
HW41	2.93E+04	0.0000	1.24E+05	3.2357	5.49E+05	17.753	4.48E+08	15302
HW42	5.29E+04	0.0000	1.64E+05	2.1021	6.01E+05	10.367	4.23E+08	8000.0
HW43	6.98E+04	0.0000	1.67E+05	1.3941	5.51E+05	6.8992	4.24E+08	6077.4
HW44	7.78E+04	0.0000	1.91E+05	1.4542	7.15E+05	8.1871	4.27E+08	5485.5
HW45	3.63E+04	0.0000	1.81E+05	3.9906	6.16E+05	15.984	4.26E+08	11744
HW46	3.09E+04	0.0000	1.76E+05	4.7049	7.98E+05	24.866	4.50E+08	14585
HW47	2.68E+04	0.0000	1.04E+05	2.8784	6.03E+05	21.487	4.24E+08	15810
HW48	1.52E+04	0.0000	1.20E+05	6.8697	7.82E+05	50.284	4.32E+08	28329
15/16			1/16		0/16		0/16	
Avg(f)			8.53E+04		3.73E+05		2.38E+08	
PDB(Avg)			2.2252		13.110		8988.1	

Table 7

Comparison results of VNSME, silver-winning, bronze-winning and baseline algorithms on the medium-scale instances of the HW benchmarks in ten runs (f_1 represents the total tardiness of orders, f_2 represents the average traveling distance of vehicles, $\lambda=10000/3600$, and PDB represents the percentage deviation from the reference value of the best solution).

instance	VNSME		Silver		Bronze		baseline	
	$f_1 + \lambda \times f_2$	PDB	$f_1 + \lambda \times f_2$	PDB	$f_1 + \lambda \times f_2$	PDB	$f_1 + \lambda \times f_2$	PDB
HW33	1.19E+03	0.000	2.75E+03	1.323	3.44E+04	27.99	1.39E+07	11722
HW34	3.44E+03	0.000	4.03E+03	0.170	1.98E+04	4.745	1.43E+07	4160
HW35	1.44E+03	0.000	5.93E+03	3.132	3.76E+04	25.15	1.84E+07	12814
HW36	5.79E+03	0.000	8.62E+03	0.489	4.09E+04	6.062	1.68E+07	2902
HW37	4.40E+03	1.087	2.11E+03	0.000	3.47E+04	15.45	1.68E+07	7983
HW38	4.59E+03	0.000	1.11E+04	1.424	3.42E+04	6.434	1.69E+07	3680
HW39	7.06E+03	0.000	8.27E+03	0.171	3.09E+04	3.379	1.60E+07	2259
HW40	3.68E+03	0.000	9.03E+03	1.454	4.24E+04	10.51	1.34E+07	3645
HW41	9.05E+03	0.000	4.51E+04	3.990	1.98E+05	20.87	1.61E+08	17828
HW42	8.82E+03	0.000	5.95E+04	5.743	2.17E+05	23.57	1.52E+08	17271
HW43	1.28E+04	0.000	6.04E+04	3.726	1.99E+05	14.53	1.53E+08	11935
HW44	1.66E+04	0.000	6.91E+04	3.166	2.58E+05	14.56	1.54E+08	9271
HW45	1.20E+04	0.000	6.57E+04	4.459	2.22E+05	17.47	1.53E+08	12754
HW46	9.97E+03	0.000	6.40E+04	5.413	2.88E+05	27.84	1.62E+08	16246
HW47	9.55E+03	0.000	3.80E+04	2.976	2.17E+05	21.76	1.53E+08	15969
HW48	5.12E+03	0.000	4.36E+04	7.514	2.82E+05	54.05	1.56E+08	30383
Avg(f)	15/16		1/16		0/16		0/16	
PDB(Avg)	7.22E+03		3.11E+04		1.35E+05		8.10E+07	
	0.000		3.305		17.662		11216.916	

reducing the average traveling distance of vehicles to save transportation costs.

As for another overall objective ($\min f = f_1 + \lambda \times f_2$), the experimental results on the medium-scale instances are illustrated in Table 7, which shows that VNSME outperforms the other compared algorithms. It is worth mentioning that, on 15 out of 16 instances, VNSME consistently found the best solutions in 10 independent runs. Comparing to the other three compared algorithms, the average performance Avg(f) of VNSME is still significantly better, which indicates that VNSME performs both strongly and robustly even if reversing the priority of minimizing f_1 and f_2 .

The remarkable advantages of VNSME are due to the fact that, VNSME has a strong ability in exploring the search space since it can gain high-diversity solutions with the four local search strategies as well as adopting large-step-size operators to escape from local optima. As a result, VNSME can usually identify more promising regions than the

compared algorithms, which has higher probability for finding superior solutions.

5.3.3. Results on large-scale HW benchmarks

In each of the large-scale instances (HW49-HW57) shown in Fig. 7, the new orders are densely distributed at 144 intervals of a day, and the number of new orders is large at most intervals, which it is difficult to find a best solution both with the smaller values of f_1 and f_2 .

As for the original overall objective in Eq. (5), our algorithm still focuses to minimize the overall objective value f of the new practical DPDP. Table 8 presents the best and average performance of VNSME and the three compared algorithms on the large-scale HW benchmarks. It is worth mentioning that, on 11 out of 16 instances, VNSME found the best solutions in 10 independent runs and performed significantly better than the other three algorithms. Besides, on the large-scale HW benchmarks, the PDB between the average performance of VNSME, the silver-winning

Table 8

Comparison results of VNSME, silver-winning, bronze-winning and baseline algorithms on the large-scale instances of the HW benchmarks in ten runs (f_1 represents the total tardiness of orders, f_2 represents the average traveling distance of vehicles, $\lambda=10000/3600$, and PDB represents the percentage deviation from the reference value of the best solution).

instance	VNSME		Silver		Bronze		baseline	
	$\lambda \times f_1 + f_2$	PDB	$\lambda \times f_1 + f_2$	PDB	$\lambda \times f_1 + f_2$	PDB	$\lambda \times f_1 + f_2$	PDB
HW49	1.58E+06	0.340	1.18E+06	0.000	1.59E+06	0.347	9.20E+08	778.6
HW50	9.31E+05	0.628	5.72E+05	0.000	2.51E+06	3.388	9.20E+08	1607
HW51	7.44E+04	0.000	5.15E+05	5.921	1.34E+06	17.00	9.13E+08	12268
HW52	7.61E+05	0.698	4.48E+05	0.000	2.03E+06	3.531	9.20E+08	2052
HW53	8.47E+04	0.000	3.17E+05	2.744	1.93E+06	21.79	9.55E+08	11279
HW54	2.07E+06	0.268	1.63E+06	0.000	2.00E+06	0.227	9.11E+08	557.8
HW55	3.23E+05	0.000	7.30E+05	1.261	1.98E+06	5.133	8.97E+08	2777
HW56	2.08E+06	0.128	1.84E+06	0.000	2.16E+06	0.174	9.45E+08	512.5
HW57	1.04E+07	0.000	2.24E+07	1.154	1.65E+07	0.587	1.83E+09	175.0
HW58	1.00E+07	0.000	1.32E+07	0.319	1.12E+07	0.119	1.81E+09	179.9
HW59	7.77E+06	0.000	1.24E+07	0.595	9.54E+06	0.227	1.90E+09	243.4
HW60	1.07E+07	0.000	2.15E+07	1.011	1.32E+07	0.235	1.83E+09	170.2
HW61	6.03E+06	0.000	9.10E+06	0.510	7.21E+06	0.197	1.81E+09	299.4
HW62	8.80E+06	0.000	1.45E+07	0.647	1.09E+07	0.238	1.87E+09	211.4
HW63	1.33E+07	0.000	3.09E+07	1.315	2.50E+07	0.873	1.91E+09	142.0
HW64	1.57E+07	0.000	2.54E+07	0.617	2.74E+07	0.744	1.96E+09	123.7
Avg(f)	11/16		5/16		0/16		0/16	
PDB(Avg)	5.67E+06		9.79E+06		8.53E+06		1.39E+09	
	0.000		0.728		0.506		245.0	

Table 9

Comparison results of VNSME, silver-winning, bronze-winning and baseline algorithms on the large-scale instances of the HW benchmarks in ten runs (f_1 represents the total tardiness of orders, f_2 represents the average traveling distance of vehicles, $\lambda=10000/3600$, and PDB represents the percentage deviation from the reference value of the best solution).

instance	VNSME		Silver		Bronze		baseline	
	$f_1 + \lambda \times f_2$	PDB	$f_1 + \lambda \times f_2$	PDB	$f_1 + \lambda \times f_2$	PDB	$f_1 + \lambda \times f_2$	PDB
HW49	4.33E+05	0.016	4.27E+05	0.000	5.74E+05	0.346	3.31E+08	775.3
HW50	1.96E+05	0.000	2.06E+05	0.052	9.05E+05	3.613	3.31E+08	1687
HW51	9.10E+04	0.000	1.86E+05	1.043	4.81E+05	4.287	3.29E+08	3614
HW52	2.21E+05	0.366	1.61E+05	0.000	7.31E+05	3.530	3.31E+08	2051
HW53	9.65E+04	0.000	1.14E+05	0.186	6.95E+05	6.206	3.44E+08	3564
HW54	9.23E+05	0.573	5.87E+05	0.000	7.18E+05	0.224	3.28E+08	557.8
HW55	1.04E+05	0.000	2.63E+05	1.526	7.11E+05	5.837	3.23E+08	3103
HW56	5.56E+05	0.000	6.63E+05	0.193	7.77E+05	0.398	3.40E+08	611.0
HW57	3.54E+06	0.000	8.08E+06	1.285	5.95E+06	0.682	6.60E+08	185.8
HW58	3.17E+06	0.000	4.77E+06	0.504	4.04E+06	0.274	6.51E+08	204.3
HW59	3.18E+06	0.000	4.45E+06	0.400	3.44E+06	0.082	6.82E+08	213.7
HW60	3.75E+06	0.000	7.75E+06	1.065	4.75E+06	0.265	6.57E+08	174.1
HW61	2.02E+06	0.000	3.27E+06	0.624	2.60E+06	0.287	6.52E+08	322.2
HW62	4.50E+06	0.151	5.24E+06	0.339	3.91E+06	0.000	6.73E+08	171.0
HW63	5.95E+06	0.000	1.11E+07	0.867	9.01E+06	0.513	6.89E+08	114.6
HW64	4.70E+06	0.000	9.13E+06	0.944	9.87E+06	1.103	7.05E+08	149.2
Avg(f)	12/16		3/16		1/16		0/16	
PDB(Avg)	2.09E+06		3.53E+06		3.07E+06		5.02E+08	
	0.000		0.688		0.471		239.172	

algorithm, the bronze-winning algorithm and the baseline are 0.000, 0.728, 0.506 and 245, respectively. Such results demonstrate that VNSME performs very stably, although the large-scale instances are more difficult. Specifically, as illustrated in Supplementary Table 5 of the supplementary material, when compared to the silver-winning, bronze-winning and baseline algorithms, VNSME found the best solutions with the smallest objective values f as well as the smallest objective values f_1 on 11 out of 16 HW instances. It is worth mentioning that the solutions obtained by the bronze-winning algorithm have smaller average traveling distances of vehicles (f_2) in all large-scale HW benchmarks (HW49-HW64), but have very large total tardiness (f_1). This phenomenon shows that there is a certain degree of conflict between minimizing f_1 and minimizing f_2 in the large-scale HW benchmarks. Therefore, VNSME prefers to balance f_1 and f_2 in order to find a solution with a smaller overall objective value f in Eq. (5), so that the solutions found by VNSME perform relatively poorly on f_2 in HW49-HW64. Meanwhile, the average value of f obtained by VNSME in all the large-scale instances is 5.67E+6, while that obtained by the silver-winning,

bronze-winning, baseline algorithms are 9.79E+6, 8.53E+6, and 1.39E+9, respectively, which reflects the effectiveness and efficiency of VNSME in solving the large-scale HW benchmarks.

As for another overall objective ($\min f = f_1 + \lambda \times f_2$), the experimental results on these large-scale instances are illustrated in Table 9, which shows that VNSME outperforms the other compared algorithms. It is worth mentioning that, on 12 out of 16 instances, VNSME consistently found the best solutions in 10 independent runs. Comparing to the other three compared algorithms, the average performance Avg(f) of VNSME is still significantly better, which exhibits that VNSME performs both strongly and robustly even if reversing the priority of minimizing f_1 and f_2 . In order to enable comparison with further solvers in the future, the reference values of the best (available) solution of the HW benchmarks for the particular λ found by all the compared algorithms are provided in Supplementary Table 7 of the supplementary material, which also can be downloaded in <https://github.com/JunchuangCai/VNSME>.

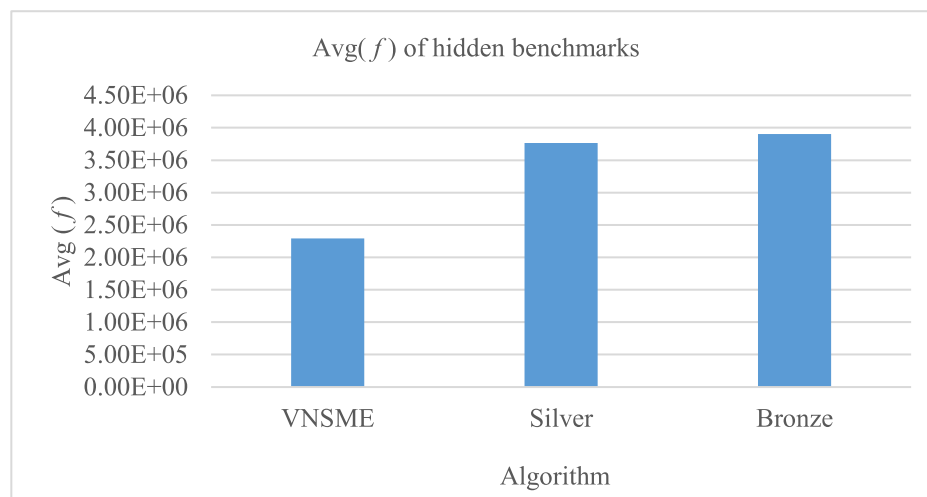


Fig. 8. Comparison of VNSME, the silver-winning algorithm and the bronze-winning algorithm based on the average cost value off on the hidden benchmarks.

5.4. Comparison results on the hidden benchmarks

In this section, we present the comparison results of running VNSME, the silver-winning algorithm and the bronze-winning algorithm on the hidden data set, similar to the HW benchmarks on the Huawei cloud platform. The platform only presents the average f value in Eq. (5) of each solution found by the submitted algorithm in the hidden data set. As a result, the average f value of VNSME in the hidden data set is $2.29\text{E}+06$, while that of the silver-winning algorithm and the Bronze-winning algorithm are $3.77\text{E}+06$ and $3.91\text{E}+06$, respectively, which are illustrated in Fig. 8. The average values of f for the solutions found by the silver-winning algorithm and the bronze-winning algorithm on the hidden data set are similar, but the average value of f obtained by VNSME is significantly better. The reason for this phenomenon is that the previous two algorithms may fall into the local optimum, while only the perturbation strategy in our algorithm can jump out of the local optimum and be able to find a better solution after applying the proposed four efficient local search strategies. The comparison experiment on the hidden benchmarks can further prove the effectiveness of VNSME.

5.5. Effectiveness of each component in VNSME

An ablation study was conducted to assess the effectiveness of the novel components integrated into VNSME. More specifically, we test the following 3 different variants of VNSME on the HW benchmark set, each of which is different from the original VNSME in one component:

- 1 CI: In initialization, it uses a hybrid mechanism consisting of exhaustion and CI method to obtain a satisfactory solution for the new DPDP variant. Specifically, the exhaustive method is used when the node of a path is less than N . The CI method is used when it is greater than N .
- 2 LS: After the CI method obtains the initial solution, the four local search strategies are used to search for the local optimal solution for the initial solution.
- 3 Disturbance: After the local search procedure, it may get trapped in the local optimum. Thus, this variant uses a slack mechanism in 2-opt-L* [45] to slack the cost of the solution obtained by LS.

For a constant parameter setting, the slackness parameter δ is uniformly set to 6, and the time for each vehicle to approach the factory's dock after arriving at the factory is 30 minutes.

The average cost of f in Eq. (5) obtained by the three variants in the 8 problem scales are shown in Fig. 9. Specifically, the three variants run 8 instances of each scale problem in the HW benchmark set, and take the average values of f for the 8 instances as the cost value of each variant in the scale problem. From the line graph of the mean f of the eight scale problems in Fig. 9, it can be clearly observed that the proposed LS strategies and the perturbation method have significant positive effects for solving the new DPDP variant. In addition to HW33-HW40, from the average cost line chart of other scale problems, it is apparent that the local search operator containing four improved local search strategies has a significant effect on the HW benchmark set. In HW33-HW40, the LS has no effect on the solution obtained by CI, which shows that the solution obtained by CI has fallen into a local optimum region, so the

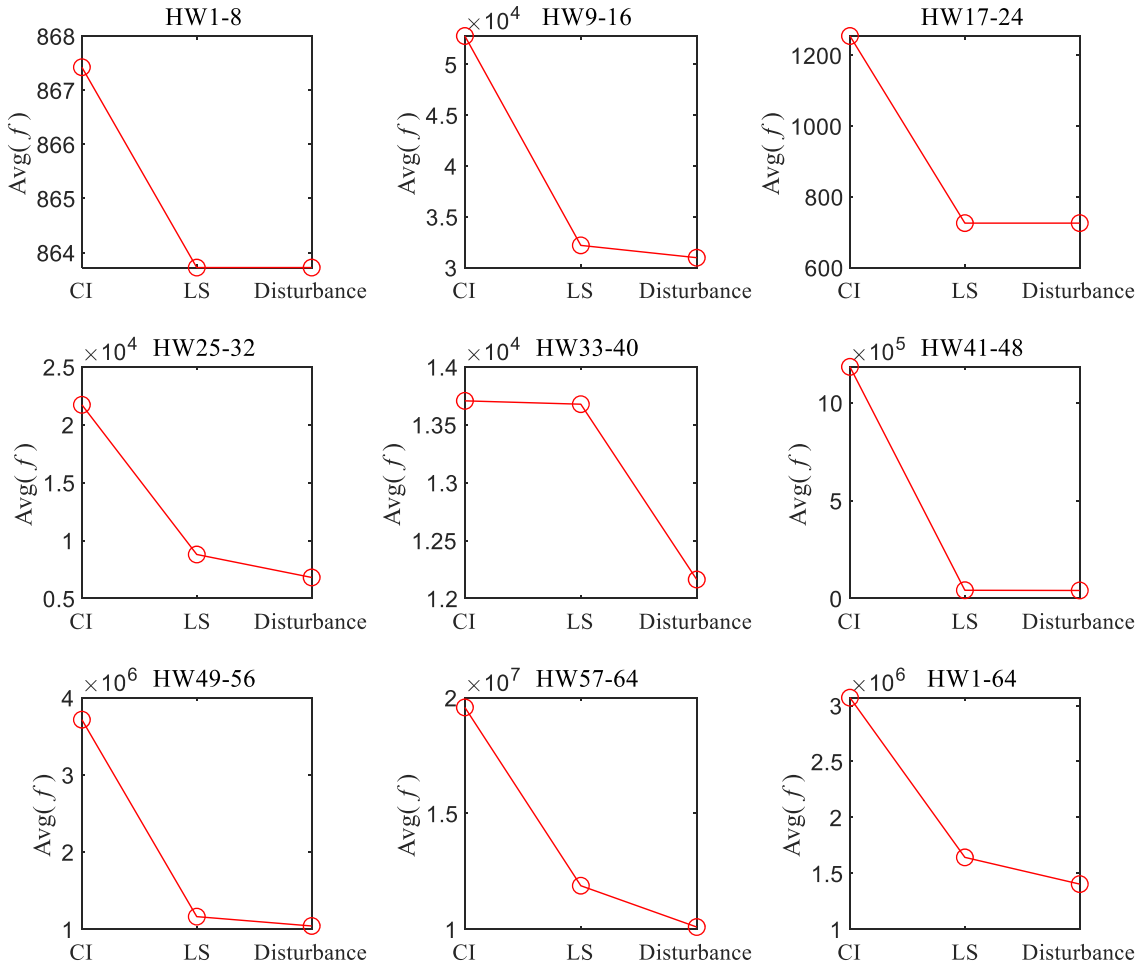


Fig. 9. The average cost of f obtained by the three variants in the 8 HW benchmark scales.

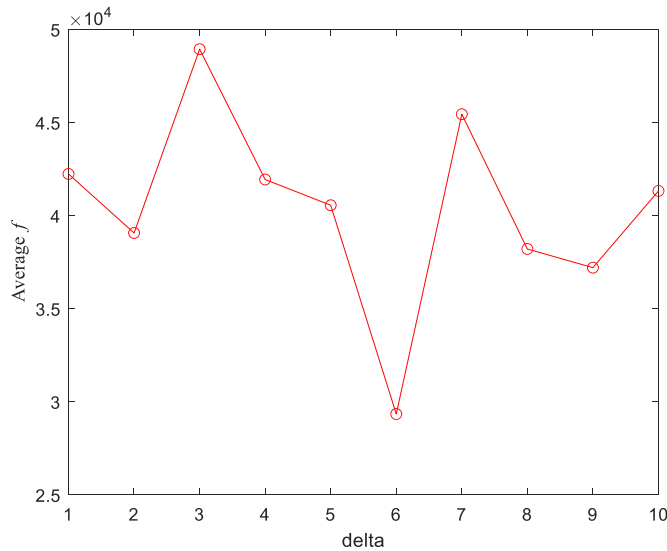


Fig. 10. Parameter sensitivity analysis of perturbation degree parameter δ .

local search has no significant effect. From the perspective of the role of the perturbation method, it can be seen from HW9-HW16, HW33-HW40 and HW57-HW64 that our perturbation method can effectively jump out from the local optimum. However, our perturbation method has no significant effect on other scale problems of the HW benchmarks, which shows that the solution obtained by the local search strategy is close to the global optimum.

5.6. Sensitivity analysis of the perturbation degree δ

The perturbation parameter δ (line 4 in Algorithm 4) is a user-defined parameter that has an important impact on the algorithm's performance. In this session, a smaller δ can make VNSME converge quickly but may easily fall into the local optimal region, while a larger δ can make VNSME jump out of the local optimal region, but it is difficult to converge to the global optimum. In other word, a smaller δ facilitates VNSME to search more sufficiently in local areas, leading to better exploitation while a larger δ directly enables more adequate coverage on the space of feasible solutions, leading to better exploration. Given a limited time budget (e.g., ten minutes), the value of δ determines the trade-off between exploration and exploitation.

To quantitatively investigate the impact of δ , we conduct ten experiments with different δ on the medium-scale instances. More specifically, we take 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 for the δ value in VNSME and conduct experiments under the same other conditions, such as the maximum running time with 600 seconds, the overall objective ($\min f = \lambda \times f_1 + f_2$) and so on. For each considered δ , its average f on the benchmark set is shown in Fig. 10. Here, a larger average value of f indicates the worse performance with the specific value of δ on the benchmarks.

Two important observations can be made from Fig. 10. First, the solution got by local search is of strict gradient descent without disturbance when δ is 1 and the average values of f with $\delta=3$ and 7 are larger than that with $\delta=1$, which indicates that $\delta=3$ and 7 may guide the search direction to a worse search space and cannot find a better solution in the maximum running time. Second, VNSME with $\delta=2$ and 9 can only obtain some local optimal solutions, while that with $\delta=6$ can find the best solution. More specifically, VNSME with $\delta=2$ modifies only a small part of x_b ; thus the neighborhoods defined by it are actually a small region around x_b , and VNSME cannot jump out of the local optimum. On the contrary, VNSME with $\delta=9$ cannot converge to the global optimal solution within the limited time. However, VNSME with $\delta=6$ can take the diversity and convergence of the solutions into account, and finally

obtains the best solution. Therefore, the above experimental results show that the parameter δ is indeed important for VNSME, and setting δ to 6 is a reasonably good default choice if no other prior knowledge is suggested.

6. Conclusions and future work

In this article, we have designed a variable neighborhood search algorithm with multiple local search strategies and an efficient disturbance to solve a new DPDP variant with sufficiently practical constraints. The four efficient local search strategies can effectively improve solutions, while the disturbance operator is able to help solutions jump out of local optimum. Our extensive comparison experiments have shown that our VNSME algorithm has significant advantages when dealing with the new DPDP variant on the HW benchmarks. After that, more ablation experiments and parameter sensitivity analysis show that VNSME can significantly improve the solutions in practice on both efficiency and effectiveness.

In the future, we plan to add the functions of the delayed delivery order strategy and the forecast order strategy to VNSME, trying to further improve its performance in this new practical DPDP. As the total tardiness of orders and the average traveling distance of vehicles are conflicting to each other, it is interesting to treat the new DPDP variant as a bi-objective optimization problem, trying to meet the more complex applications of DPDP in reality. Meanwhile, the design of a hybrid algorithm to better solve the bi-objective optimization problem can be another further research direction.

CRediT authorship contribution statement

Junchuang Cai: Conceptualization, Methodology, Software. **Qingling Zhu:** Writing – original draft. **Qiuzhen Lin:** Writing – review & editing, Supervision.

Declaration of competing interest

None.

Acknowledgements

This work was supported by the National Natural Science Foundation of China (NSFC) under Grants 61876110 and 61836005, the Joint Funds of the NSFC under Key Program Grant U1713212, and by the Shenzhen Scientific Research and Development Funding Program under grant JCYJ20190808164211203, the Guangdong “Pearl River Talent Recruitment Program” under Grant 2019ZT08×603, and Shenzhen Science and Technology Innovation Commission (R2020A045).

Supplementary materials

Supplementary material associated with this article can be found, in the online version, at doi:[10.1016/j.swevo.2022.101182](https://doi.org/10.1016/j.swevo.2022.101182).

References

- [1] J.-F. Cordeau, G. Laporte, and S. Ropke, “Recent Models and Algorithms for One-to-One Pickup and Delivery Problems,” pp. 327–357: Springer US.
- [2] V. Nenchev, C.G. Cassandras, J. Raisch, Event-driven optimal control for a robotic exploration, pick-up and delivery problem, *Nonlinear Anal. Hybrid Syst.* 30 (2018) 266–284.
- [3] E. Osaba, X.-S. Yang, I. Fister Jr, J. Del Ser, P. Lopez-Garcia, A.J. Vazquez-Pardavila, e. computation, A discrete and improved bat algorithm for solving a medical goods distribution problem with pharmacological waste collection, *Swarm. Evol. Comput.* 44 (2019) 273–286.
- [4] G. Berbeglia, J.-F. Cordeau, G. Laporte, Dynamic pickup and delivery problems, *Eur. J. Oper. Res.* 202 (1) (2010) 8–15.
- [5] K. Treleaven, M. Pavone, E. Frazzoli, Asymptotically optimal algorithms for one-to-one pickup and delivery problems with applications to transportation systems, *IEEE Trans. Autom. Control* 58 (9) (2013) 2261–2276.

- [6] G. Berbeglia, J.-F. Cordeau, I. Gribkovskaia, G. Laporte, Static pickup and delivery problems: a classification scheme and survey, *TOP* 15 (1) (2007) 1–31.
- [7] S. Liu, K. Tang, X. Yao, Memetic search for vehicle routing with simultaneous pickup-delivery and time windows, *Swarm Evol. Comput.* 66 (2021), 100927.
- [8] H.N. Psaraftis, M. Wen, C.A. Kontovas, Dynamic vehicle routing problems: Three decades and counting, *Networks* 67 (1) (2016) 3–31.
- [9] S. Mitrović-Minić, G. Laporte, Waiting strategies for the dynamic pickup and delivery problem with time windows, *Transp. Res. Part B* 38 (7) (2004) 635–655.
- [10] M. Gendreau, F. Guertin, J.-Y. Potvin, R. Séguin, Neighborhood search heuristics for a dynamic vehicle dispatching problem with pick-ups and deliveries, *Transp. Res. Part C: Emerg. Technol.* 14 (3) (2006) 157–174.
- [11] D. Sáez, C.E. Cortés, A. Núñez, Hybrid adaptive predictive control for the multi-vehicle dynamic pick-up and delivery problem based on genetic algorithms and fuzzy clustering, *Appl. Comput. Oper. Res. Miner. Ind., Proc. Int. Symp., 30th* 35 (11) (2008) 3412–3438.
- [12] V. Pureza, G. Laporte, Waiting and buffering strategies for the dynamic pickup and delivery problem with time windows, *INFOR: Inf. Syst. Oper. Res.* 46 (3) (2008) 165–175.
- [13] X. Li, W. Luo, M. Yuan, J. Wang, J. Lu, J. Wang, J. Lü, J. Zeng, Learning to optimize industry-scale dynamic pickup and delivery problems, in: 2021 IEEE 37th International Conference on Data Engineering (ICDE), IEEE, 2021, pp. 2511–2522.
- [14] M. Savelsbergh, M. Sol, Drive: Dynamic routing of independent vehicles, *Oper. Res.* 46 (4) (1998) 474–490.
- [15] A. Bombelli, S. Fazi, The ground handler dock capacitated pickup and delivery problem with time windows: a collaborative framework for air cargo operations, *Transp. Res. E Logist. Transp. Rev.* 159 (2022), 102603.
- [16] J. Hao, J. Lu, X. Li, X. Tong, X. Xiang, M. Yuan, and H. H. Zhuo, “Introduction to the dynamic pickup and delivery problem benchmark-ICAPS 2021 competition,” *arXiv preprint arXiv:2202.01256*, 2022.
- [17] Q. Lu, M. Dessouky, An exact algorithm for the multiple vehicle pickup and delivery problem, *Transp. Sci.* 38 (4) (2004) 503–514.
- [18] M. Mahmoudi, X. Zhou, Finding optimal solutions for vehicle routing problem with pickup and delivery services with time windows: a dynamic programming approach based on state-space-time network representations, *Transp. Res. Part B: Methodol.* 89 (2016) 19–42.
- [19] J. Euchi, R. Mrahihi, The urban bus routing problem in the Tunisia case by the hybrid artificial ant colony algorithm, *Swarm Evol. Comput.* 2 (2012) 15–24.
- [20] J. Euchi, A. Yassine, H. Chabchoub, The dynamic vehicle routing problem: Solution with hybrid metaheuristic approach, *Swarm Evol. Comput.* 21 (2015) 41–53.
- [21] L.P. Cota, F.G. Guimarães, R.G. Ribeiro, I.R. Meneghini, F.B. de Oliveira, M. J. Souza, P. Siarry, E. Computation, An adaptive multi-objective algorithm based on decomposition and large neighborhood search for a green machine scheduling problem, *Swarm Evol. Comput.* 51 (2019), 100601.
- [22] K.V. Dasari, V. Pandiri, A. Singh, Multi-start heuristics for the profitable tour problem, *Swarm. Evol. Comput.* 64 (2021), 100897.
- [23] D. Trachanatzi, M. Rigakis, M. Marinaki, Y. Marinakis, A firefly algorithm for the environmental prize-collecting vehicle routing problem, *Swarm. Evol. Comput.* 57 (2020), 100712.
- [24] N.R. Sabar, A. Bhaskar, E. Chung, A. Turkey, A. Song, A self-adaptive evolutionary algorithm for dynamic vehicle routing problems with traffic congestion, *Swarm. Evol. Comput.* 44 (2019) 1018–1027.
- [25] M. Okulewicz, J. Mańdziuk, A metaheuristic approach to solve Dynamic Vehicle Routing Problem in continuous search space, *Swarm. Evol. Comput.* 48 (2019) 44–61.
- [26] İ. İlhan, An improved simulated annealing algorithm with crossover operator for capacitated vehicle routing problem, *Swarm. Evol. Comput.* 64 (2021), 100911.
- [27] S.S. Choong, L.-P. Wong, C.P. Lim, An artificial bee colony algorithm with a modified choice function for the traveling salesman problem, *Swarm. Evol. Comput.* 44 (2019) 622–635.
- [28] Y. Marinakis, M. Marinaki, E. Computation, A bumble bees mating optimization algorithm for the open vehicle routing problem, *Swarm. Evol. Comput.* 15 (2014) 80–94.
- [29] W. Dong, K. Zhou, H. Qi, C. He, J. Zhang, A tissue P system based evolutionary algorithm for multi-objective VRPTW, *Swarm. Evol. Comput.* 39 (2018) 310–322.
- [30] Y. Wang, L. Wang, Z. Peng, G. Chen, Z. Cai, L. Xing, A multi ant system based hybrid heuristic algorithm for vehicle routing problem with service time customization, *Swarm. Evol. Comput.* 50 (2019), 100563.
- [31] S. Iqbal, M. Kaykobad, M.S. Rahman, Solving the multi-objective vehicle routing problem with soft time windows with the help of bees, *Swarm. Evol. Comput.* 24 (2015) 50–64.
- [32] E. Osaba, E. Villar-Rodríguez, J.D. Ser, A.J. Nebro, D. Molina, A. LaTorre, P. N. Suganthan, C.A. Coello, F. Herrera, A Tutorial On the design, experimentation and application of metaheuristic algorithms to real-World optimization problems, *Swarm. Evol. Comput.* 64 (2021), 100888.
- [33] M.R. Swihart, J.D. Papastavrou, A stochastic and dynamic model for the single-vehicle pick-up and delivery problem, *Eur. J. Oper. Res.* 114 (3) (1999) 447–464.
- [34] S. Mitrović-Minić, R. Krishnamurti, G. Laporte, Double-horizon based heuristics for the dynamic pickup and delivery problem with time windows, *Transp. Res. Part B* 38 (8) (2004) 669–685.
- [35] S. Vonolfen, M. Affenzeller, Distribution of waiting time for dynamic pickup and delivery problems, *Ann. Oper. Res.* 236 (2) (2016) 359–382.
- [36] K. Fagerholt, B. Foss, O. Horgen, A decision support model for establishing an air taxi service: a case study, *J. Oper. Res. Soc.* 60 (9) (2009) 1173–1182.
- [37] M. Schilde, K.F. Doerner, R.F. Hartl, Metaheuristics for the dynamic stochastic dial-a-ride problem with expected return transports, *Comput. Oper. Res.* 38 (12) (2011) 1719–1730.
- [38] M. Schilde, K.F. Doerner, R.F. Hartl, Integrating stochastic time-dependent travel speed in solution methods for the dynamic dial-a-ride problem, *Eur. J. Oper. Res.* 238 (1) (2014) 18–30.
- [39] D. Reyes, A. Erera, M. Savelsbergh, S. Sahasrabudhe, R. O’Neil, The meal delivery routing problem, *Optim. Online* (2018).
- [40] M.W. Ulmer, B.W. Thomas, A.M. Campbell, N. Woyak, The restaurant meal delivery problem: dynamic pickup and delivery with deadlines and random ready times, *Transp. Sci.* 55 (1) (2021) 75–100.
- [41] G. Ghiani, E. Manni, A. Quaranta, C. Triki, T. Review, Anticipatory algorithms for same-day courier dispatching, *Transp. Res. E: Logist. Transp. Rev.* 45 (1) (2009) 96–106.
- [42] D. Muñoz-Carpintero, D. Sáez, C.E. Cortés, A. Núñez, A methodology based on evolutionary algorithms to solve a dynamic pickup and delivery problem under a hybrid predictive control approach, *Transp. Sci.* 49 (2) (2015) 239–253.
- [43] F. Karami, W. Vancroonenburg, G. Berghé, A periodic optimization approach to dynamic pickup and delivery problems with time windows, *J. Sched.* 23 (6) (2020) 711–731.
- [44] L. Cassani, G. Righini, Heuristic algorithms for the TSP with rear-loading, in: 35th Annual Conference of the Italian Operational Research Society (AIRO XXXV), Lecce, 2004.
- [45] F. Carrabs, J.-F. Cordeau, G. Laporte, Variable neighborhood search for the pickup and delivery traveling salesman problem with LIFO loading, *INFORMS J. Comput.* 19 (4) (2007) 618–632.
- [46] <https://competition.huaweicloud.com/information/1000041411/circumstance>.
- [47] <https://competition.huaweicloud.com/information/1000041411/Winning>.
- [48] https://github.com/huawei-noah/xingtian/tree/master/simulator/dpdp_competition.